Финансовый университет при Правительстве Российской Федерации

Факультет информационных технологий и анализа больших данных

Направление подготовки 10.03.01 «Информационная безопасность»

ОТЧЁТ

по результатам анализа защищённости веб-приложения
OWASP Juice Shop

(поиск, эксплуатация и устранение уязвимостей)

Часть 2. Дополнительные уязвимости

Выполнил: студент группы ИБ24-5
Шишкин Кирилл Владимирович

Москва 2026

СОДЕРЖАНИЕ

1. Введение.....5
2. Этап разведки.....6
3. Обнаруженные уязвимости.....12
3.1. Уязвимость №1. CSRF: смена пароля через межсайтовую подделку запроса.....12
3.1.1. Как нашли.....12
3.1.2. Как использовать.....12
3.1.3. Как исправить.....13
3.2. Уязвимость №2. Открытое перенаправление (Open Redirect).....14
3.2.1. Как нашли.....14
3.2.2. Как использовать.....14
3.2.3. Как исправить.....15
3.3. Уязвимость №3. Небезопасная конфигурация CORS.....16
3.3.1. Как нашли.....16
3.3.2. Как использовать.....16
3.3.3. Как исправить.....16
3.4. Уязвимость №4. Загрязнение параметров HTTP (HTTP Parameter Pollution).....18
3.4.1. Как нашли.....18
3.4.2. Как использовать.....18
3.4.3. Как исправить.....18
3.5. Уязвимость №5. Обход клиентской защиты от XSS.....19
3.5.1. Как нашли.....20
3.5.2. Как использовать.....20
3.5.3. Как исправить.....20
3.6. Уязвимость №6. Обход серверной защиты от XSS.....21
3.6.1. Как нашли.....22
3.6.2. Как использовать.....22
3.6.3. Как исправить.....22
3.7. Уязвимость №7. Хранимый XSS только через API (без фронтенда)23
3.7.1. Как нашли.....23
3.7.2. Как использовать.....24
3.7.3. Как исправить.....24
3.8. Уязвимость №8. Бизнес-логика: отрицательное количество в корзине 25

3.8.1. Как нашли.....25
3.8.2. Как использовать.....25
3.8.3. Как исправить.....25
3.9. Уязвимость №9. Бизнес-логика: повторное использование купона / реферальный фрод.....27
3.9.1. Как нашли.....27
3.9.2. Как использовать.....27
3.9.3. Как исправить.....27
3.10. Уязвимость №10. Бизнес-логика: манипуляция балансом кошелька29
3.10.1. Как нашли.....29
3.10.2. Как использовать.....29
3.10.3. Как исправить.....29
3.11. Уязвимость №11. Получение Deluxe-членства без оплаты.....31
3.11.1. Как нашли.....31
3.11.2. Как использовать.....31
3.11.3. Как исправить.....31
3.12. Уязвимость №12. Перечисление пользователей (user enumeration)32
3.12.1. Как нашли.....33
3.12.2. Как использовать.....33
3.12.3. Как исправить.....33
3.13. Уязвимость №13. JWT без срока действия (токен не истекает).....34
3.13.1. Как нашли.....35
3.13.2. Как использовать.....35
3.13.3. Как исправить.....35
3.14. Уязвимость №14. Предсказуемый пароль OAuth-аккаунта.....36
3.14.1. Как нашли.....37
3.14.2. Как использовать.....37
3.14.3. Как исправить.....37
3.15. Уязвимость №15. Нарушение приватности: доступ к данным и удалению чужого аккаунта.....38
3.15.1. Как нашли.....39
3.15.2. Как использовать.....39
3.15.3. Как исправить.....39
3.16. Уязвимость №16. Раскрытие внутренних метрик (/metrics).....40
3.16.1. Как нашли.....40
3.16.2. Как использовать.....40

3.16.3. Как исправить.....40
3.17. Уязвимость №17. Устаревший B2B-интерфейс обработки XML..42
3.17.1. Как нашли.....42
3.17.2. Как использовать.....42
3.17.3. Как исправить.....42
3.18. Уязвимость №18. Чтение произвольных файлов (path traversal в API) 43
3.18.1. Как нашли.....44
3.18.2. Как использовать.....44
3.18.3. Как исправить.....44
3.19. Уязвимость №19. Отказ в обслуживании через разбор YAML (YAML-бомба).....45
3.19.1. Как нашли.....46
3.19.2. Как использовать.....46
3.19.3. Как исправить.....46
3.20. Уязвимость №20. Смена пароля без текущего пароля.....47
3.20.1. Как нашли.....48
3.20.2. Как использовать.....48
3.20.3. Как исправить.....48
3.21. Уязвимость №21. Раскрытие информации через подробные сообщения об ошибках.....49
3.21.1. Как нашли.....49
3.21.2. Как использовать.....49
3.21.3. Как исправить.....50
3.22. Уязвимость №22. Слабая политика паролей.....51
3.22.1. Как нашли.....51
3.22.2. Как использовать.....51
3.22.3. Как исправить.....51
4. Заключение.....52
5. Список литературы.....53

1. Введение

Настоящий документ — вторая часть отчёта по анализу защищённости приложения OWASP Juice Shop. В первой части были разобраны 27 уязвимостей (SQL/NoSQL-инъекции, XSS, нарушения контроля доступа, криптографические недостатки, XXE, SSRF и др.). В этой части рассматриваются дополнительные уязвимости, не вошедшие в первую: межсайтовая подделка запроса (CSRF), открытое перенаправление, небезопасный CORS, дефекты бизнес-логики (манипуляция количеством, кошельком, членством), перечисление пользователей, бессрочные JWT, чтение произвольных файлов, отказ в обслуживании через разбор YAML и другие.

OWASP Juice Shop — умышленно уязвимое веб-приложение под эгидой OWASP (Open Worldwide Application Security Project), предназначенное для обучения тестированию защищённости. Приложение построено на стеке Node.js, Express и Angular, использует ORM Sequelize поверх СУБД SQLite и MongoDB-совместимое хранилище MarsDB, и содержит набор преднамеренно внедрённых уязвимостей, покрывающих рейтинг OWASP Top 10 2021.

Теоретическую основу составляют материалы проекта OWASP Juice Shop[1], официальное руководство Pwning OWASP Juice Shop[2] и классификация OWASP Top 10[3].

Цель работы — провести анализ защищённости приложения: выполнить разведку, обнаружить максимально возможное число уязвимостей, продемонстрировать их эксплуатацию и предложить способы устранения. Все действия выполнялись на локальном учебном стенде: серверная часть Juice Shop развёрнута на хосте Windows 10, атакующая машина — Kali Linux.

[1] Электронный ресурс [https://owasp.org/www-project-juice-shop/] — OWASP Juice Shop Project, OWASP Foundation, дата обращения: 15.09.2026.
[2] Электронный ресурс [https://pwning.owasp-juice.shop/] — Pwning OWASP Juice Shop (companion guide), Björn Kimminich, дата обращения: 15.09.2026.
[3] Электронный ресурс [https://owasp.org/www-project-top-ten/] — OWASP Top 10, OWASP Foundation, дата обращения: 16.09.2026.

Тестирование умышленно уязвимого приложения в изолированной среде легально и этично.

Отчёт построен по трём этапам задания: разведка, обнаружение уязвимостей и их устранение. Для каждой уязвимости приведены способ обнаружения, практическая эксплуатация и развёрнутые рекомендации по исправлению с рекомендуемой моделью, кодом «было → стало» и поясняющей схемой «как неправильно / как надо». Надстрочные сноски дублируются внизу страницы и соответствуют нумерации в списке литературы.

2. Этап разведки

Общая разведка приложения была выполнена на этапе первой части отчёта (анализ бандла main.js, карта REST API, перебор путей, идентификация версий и ролевой модели). Для уязвимостей этой части разведка была дополнена прицельными проверками, перечисленными ниже.

Анализ заголовков ответов. Через curl -I и вкладку Network изучены CORS-заголовки (Access-Control-Allow-Origin, Allow-Credentials), заголовки безопасности и сообщения об ошибках, что выявило отражение Origin и утечку стектрейсов.

Разбор изменяющих операций. Для форм смены пароля, пополнения кошелька, апгрейда членства и работы с корзиной зафиксированы HTTP-методы, параметры и наличие (отсутствие) anti-CSRF токенов и серверных проверок.

Проверка служебных и устаревших эндпоинтов. Перебором путей найдены /metrics (Prometheus), эндпоинт перенаправления /redirect, устаревший B2B-интерфейс приёма XML и точки отдачи файлов.

Анализ токенов и восстановления доступа. Токены JWT разобраны на jwt.io (срок действия, алгоритм); сопоставлены ответы форм входа и восстановления для оценки перечисления пользователей.

Проверка бизнес-логики. Изучены границы значений (количество товара, суммы, применение купонов) и порядок проведения платежей для поиска логических дефектов.

Итогом стала уточнённая карта дополнительных точек входа, на основе которой проверялись уязвимости этой части отчёта.

Краткое резюме обнаруженных уязвимостей

Ниже — краткие описания (что эксплуатирует, как активировать, как быстро исправить). Надстрочная сноска у каждой уязвимости ведёт к источнику внизу страницы и в списке литературы. Подробный разбор — в разделе 3.

№1. CSRF: смена пароля через межсайтовую подделку запроса [A01:2021 – Broken Access Control]. Эксплуатирует отсутствие anti-CSRF токена на операции смены пароля (GET-запрос). Активируется вредоносной страницей, выполняющей запрос от имени жертвы. Исправляется CSRF-токенами и SameSite-куками.[4]

№2. Открытое перенаправление (Open Redirect) [A01:2021 – Broken Access Control]. Эксплуатирует слабую проверку адреса в параметре redirect. Активируется ссылкой ?to=... с обходом белого списка. Исправляется строгим allow-list и относительными путями.[5]

№3. Небезопасная конфигурация CORS [A05:2021 – Security Misconfiguration]. Эксплуатирует отражение Origin и Allow-Credentials.

[4] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html] — Cross-Site Request Forgery Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 17.09.2026.

[5] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html] — Unvalidated Redirects and Forwards Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 19.09.2026.

Активируется кросс-доменным запросом с чужого сайта. Исправляется строгим белым списком Origin без credentials при '*'.[6]

№4. Загрязнение параметров HTTP (HTTP Parameter Pollution) [A03:2021 – Injection]. Эксплуатирует неоднозначную обработку повторяющихся параметров. Активируется дублированием параметра в запросе. Исправляется явной нормализацией и валидацией параметров.[7]

№5. Обход клиентской защиты от XSS [A03:2021 – Injection (XSS)]. Эксплуатирует наличие только клиентской фильтрации XSS. Активируется отправкой payload напрямую в API, минуя фронтенд. Исправляется серверной санитизацией и экранированием вывода.[8]

№6. Обход серверной защиты от XSS [A03:2021 – Injection (XSS)]. Эксплуатирует неполную серверную санитизацию (blacklist). Активируется payload, не попадающим под чёрный список. Исправляется проверенным санитайзером (allow-list) и экранированием.[9]

№7. Хранимый XSS только через API (без фронтенда) [A03:2021 – Injection (XSS)]. Эксплуатирует отсутствие валидации на REST-эндпоинте. Активируется PUT/POST с XSS в поле, которое фронтенд не позволяет ввести. Исправляется серверной валидацией и экранированием при выводе.[10]

№8. Бизнес-логика: отрицательное количество в корзине [A04:2021 – Insecure Design]. Эксплуатирует отсутствие проверки диапазона количества.

---

[6] Электронный ресурс [https://portswigger.net/web-security/cors] — Cross-origin resource sharing (CORS), PortSwigger Web Security Academy, дата обращения: 20.09.2026.
[7] Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution] — Testing for HTTP Parameter Pollution, OWASP WSTG, дата обращения: 21.09.2026.
[8] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html] — Cross Site Scripting Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 23.09.2026.
[9] Электронный ресурс [https://owasp.org/www-community/xss-filter-evasion-cheatsheet] — XSS Filter Evasion Cheat Sheet, OWASP Community, дата обращения: 24.09.2026.
[10] Электронный ресурс [https://owasp.org/www-community/Types_of_Cross-Site_Scripting] — Types of Cross-Site Scripting, OWASP Community, дата обращения: 25.09.2026.

Активируется установкой отрицательного quantity в корзине. Исправляется серверной валидацией (quantity ≥ 1, целое).[11]

№9. Бизнес-логика: повторное использование купона / реферальный фрод [A04:2021 – Insecure Design]. Эксплуатирует отсутствие учёта однократности купона. Активируется многократным применением одного кода. Исправляется серверным учётом использований и атомарной проверкой.[12]

№10. Бизнес-логика: манипуляция балансом кошелька [A01:2021 – Broken Access Control]. Эксплуатирует доверие к сумме пополнения из запроса. Активируется подменой amount при пополнении кошелька. Исправляется проведением платежа на сервере и сверкой с платёжным провайдером.[13]

№11. Получение Deluxe-членства без оплаты [A01:2021 – Broken Access Control]. Эксплуатирует проверку статуса оплаты на клиенте. Активируется прямым запросом к upgrade-эндпоинту. Исправляется серверной проверкой факта оплаты.[14]

№12. Перечисление пользователей (user enumeration) [A07:2021 – Identification and Authentication Failures]. Эксплуатирует различие ответов для существующих и несуществующих аккаунтов. Активируется сравнением откликов login/forgot-password. Исправляется единообразными ответами и таймингом.[15]

---

[11] Электронный ресурс [https://owasp.org/Top10/A04_2021-Insecure_Design/] — A04:2021 – Insecure Design, OWASP Foundation, дата обращения: 27.09.2026.
[12] Электронный ресурс [https://cwe.mitre.org/data/definitions/840.html] — CWE-840: Business Logic Errors, MITRE CWE, дата обращения: 28.09.2026.
[13] Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/README] — Business Logic Testing, OWASP WSTG, дата обращения: 29.09.2026.
[14] Электронный ресурс [https://owasp.org/Top10/A01_2021-Broken_Access_Control/] — A01:2021 – Broken Access Control, OWASP Foundation, дата обращения: 01.10.2026.
[15] Электронный ресурс [https://cwe.mitre.org/data/definitions/204.html] — CWE-204: Observable Response Discrepancy, MITRE CWE, дата обращения: 02.10.2026.

№13. JWT без срока действия (токен не истекает) [A07:2021 – Identification and Authentication Failures]. Эксплуатирует отсутствие exp и серверной инвалидации токена. Активируется повторным использованием перехваченного JWT. Исправляется коротким сроком жизни, ревокацией и ротацией.[16]

№14. Предсказуемый пароль OAuth-аккаунта [A07:2021 – Identification and Authentication Failures]. Эксплуатирует вывод пароля из e-mail обратимым кодированием (base64). Активируется восстановлением пароля по email. Исправляется криптослучайными паролями и отказом от выводимых секретов.[17]

№15. Нарушение приватности: доступ к данным и удалению чужого аккаунта [A01:2021 – Broken Access Control]. Эксплуатирует отсутствие проверки владельца в операциях экспорта/удаления данных. Активируется запросом с подменой идентификатора. Исправляется привязкой операции к текущему пользователю.[18]

№16. Раскрытие внутренних метрик (/metrics) [A05:2021 – Security Misconfiguration]. Эксплуатирует открытый эндпоинт мониторинга Prometheus. Активируется обращением к /metrics. Исправляется ограничением доступа к служебным эндпоинтам.[19]

№17. Устаревший B2B-интерфейс обработки XML [A06:2021 – Vulnerable and Outdated Components]. Эксплуатирует устаревший XML-интерфейс с опасным парсером. Активируется отправкой специально

[16] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html] — JSON Web Token Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 03.10.2026.
[17] Электронный ресурс [https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/] — A07:2021 – Identification and Authentication Failures, OWASP Foundation, дата обращения: 05.10.2026.
[18] Электронный ресурс [https://cwe.mitre.org/data/definitions/359.html] — CWE-359: Exposure of Private Personal Information, MITRE CWE, дата обращения: 06.10.2026.
[19] Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/README] — Information Gathering, OWASP WSTG, дата обращения: 08.10.2026.

сформированного XML. Исправляется выводом из эксплуатации и безопасным парсингом.[20]

№18. Чтение произвольных файлов (path traversal в API) [A01:2021 – Broken Access Control]. Эксплуатирует подстановку пути в параметр файла. Активируется последовательностями ../ в имени файла. Исправляется нормализацией пути и canonical-проверкой внутри каталога.[21]

№19. Отказ в обслуживании через разбор YAML (YAML-бомба) [A04:2021 – Insecure Design]. Эксплуатирует небезопасный разбор YAML с якорями/алиасами. Активируется загрузкой YAML с рекурсивными ссылками. Исправляется safe-парсером, лимитами и таймаутами.[22]

№20. Смена пароля без текущего пароля [A07:2021 – Identification and Authentication Failures]. Эксплуатирует отсутствие проверки текущего пароля при смене. Активируется запросом смены без поля current. Исправляется обязательной проверкой текущего пароля.[23]

№21. Раскрытие информации через подробные сообщения об ошибках [A05:2021 – Security Misconfiguration]. Эксплуатирует вывод стектрейсов и SQL-ошибок клиенту. Активируется некорректным вводом, вызывающим исключение. Исправляется общими сообщениями и серверным логированием деталей.[24]

№22. Слабая политика паролей [A07:2021 – Identification and Authentication Failures]. Эксплуатирует отсутствие требований к сложности

[20] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/XML_Security_Cheat_Sheet.html] — XML Security Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 09.10.2026.
[21] Электронный ресурс [https://owasp.org/www-community/attacks/Path_Traversal] — Path Traversal, OWASP Community, дата обращения: 11.10.2026.
[22] Электронный ресурс [https://cwe.mitre.org/data/definitions/400.html] — CWE-400: Uncontrolled Resource Consumption, MITRE CWE, дата обращения: 12.10.2026.
[23] Электронный ресурс [https://cwe.mitre.org/data/definitions/620.html] — CWE-620: Unverified Password Change, MITRE CWE, дата обращения: 13.10.2026.
[24] Электронный ресурс [https://cwe.mitre.org/data/definitions/209.html] — CWE-209: Generation of Error Message Containing Sensitive Information, MITRE CWE, дата обращения: 15.10.2026.

пароля. Активируется регистрацией с тривиальным паролем. Исправляется требованиями к длине/сложности и проверкой по базам утечек.[25]

3. Обнаруженные уязвимости

Каждая уязвимость разобрана по трём пунктам: как нашли, как использовать, как исправить. Эксплуатация описана для стенда «Windows 10 (Juice Shop) + Kali Linux».

Уязвимость №1. CSRF: смена пароля через межсайтовую подделку запроса

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.1.1. Как нашли

Смена пароля выполняется запросом GET /rest/user/change-password? new=...&repeat=..., при этом не передаётся и не проверяется никакой anti-CSRF токен, а аутентификация завязана на автоматически отправляемых данных сессии. Использование GET для изменяющей операции и отсутствие CSRF-защиты — прямой индикатор уязвимости межсайтовой подделки запроса.

3.1.2. Как использовать

Готовится вредоносная страница, которая при посещении жертвой (уже вошедшей в Juice Shop) автоматически инициирует запрос на смену пароля:

```
<img src="http://<IP>:3000/rest/user/change-password?
new=hacked123&repeat=hacked123">
```

Браузер жертвы отправляет запрос с её сессией, и пароль меняется без её ведома. Ссылка на такую страницу рассылается методами социальной инженерии.

[25] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html] — Authentication Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 16.10.2026.

3.1.3. Как исправить

Рекомендуемая модель: Anti-CSRF токены (synchronizer/double-submit) + SameSite-куки + POST для изменяющих операций.

CSRF возможен, потому что изменяющая операция выполняется по GET и аутентифицируется автоматически отправляемыми браузером данными сессии, без доказательства, что запрос инициирован самим приложением. Модель: изменяющие действия — только POST/PUT; к каждому привязывать непредсказуемый CSRF-токен (synchronizer token или double-submit cookie), который проверяется на сервере; куки сессии помечать SameSite=Strict/Lax и HttpOnly. Для API на токенах — требовать токен в заголовке (браузер не добавит его кросс-доменно автоматически) и проверять Origin/Referer.

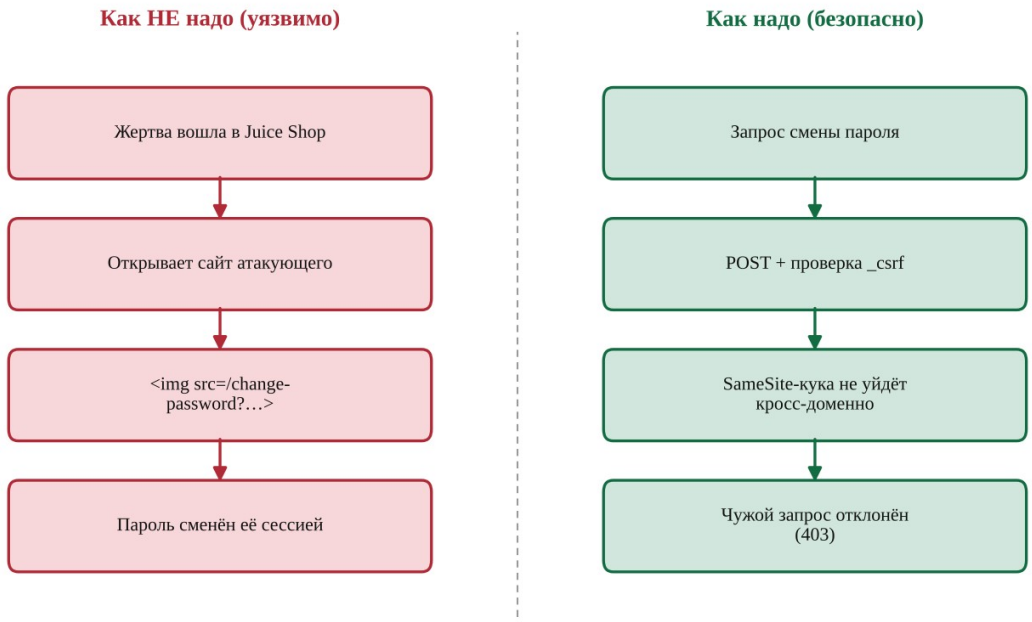


Рисунок 1 — Уязвимый и безопасный поток обработки (CSRF: смена пароля через межсайтовую подделку запроса)

Было (уязвимо):

```
// УЯЗВИМО: смена пароля по GET, без CSRF-токена
app.get('/rest/user/change-password', (req, res) => {
  const { new: n, repeat } = req.query
  if (n === repeat) setPassword(req.user, n)   // достаточно куки сессии
```

```
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: POST + CSRF-токен + SameSite
import csurf from 'csurf'
app.use(cookieParser())
const csrf = csurf({ cookie: { httpOnly: true, sameSite: 'strict' } })

app.post('/rest/user/change-password', csrf, (req, res) => {
  // csurf сам отклонит запрос без валидного _csrf
  const { current, new: n, repeat } = req.body
  if (!verifyCurrent(req.user, current)) return res.status(401).end()
  if (n === repeat) setPassword(req.user, n)
})
// куки сессии: res.cookie('token', jwt, { httpOnly:true,
sameSite:'strict', secure:true })
```

Уязвимость №2. Открытое перенаправление (Open Redirect)

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Средняя.

3.2.1. Как нашли

Эндпоинт /redirect?to=<url> перенаправляет пользователя по переданному адресу. Проверка реализована как поиск доверенной подстроки в URL, что обходится: достаточно включить разрешённый адрес как параметр внутри вредоносной ссылки.

3.2.2. Как использовать

Формируется ссылка, где белый список обманывается включением доверенного URL в query-часть вредоносного:

```
http://<IP>:3000/redirect?to=https://evil.example.com/?x=https://
github.com/juice-shop/juice-shop
```

Проверка находит доверенную подстроку, но фактическое перенаправление уходит на сайт атакующего — используется для фишинга.

3.2.3. Как исправить

Рекомендуемая модель: Белый список целей + относительные пути (отказ от произвольных URL).

Проверка «содержит доверенную подстроку» обходится, так как доверенный URL можно вложить в параметр вредоносного. Модель: не перенаправлять по произвольному внешнему URL; разрешать только заранее известные относительные пути (allow-list) или сопоставлять по точному хосту после разбора URL, а не по подстроке.

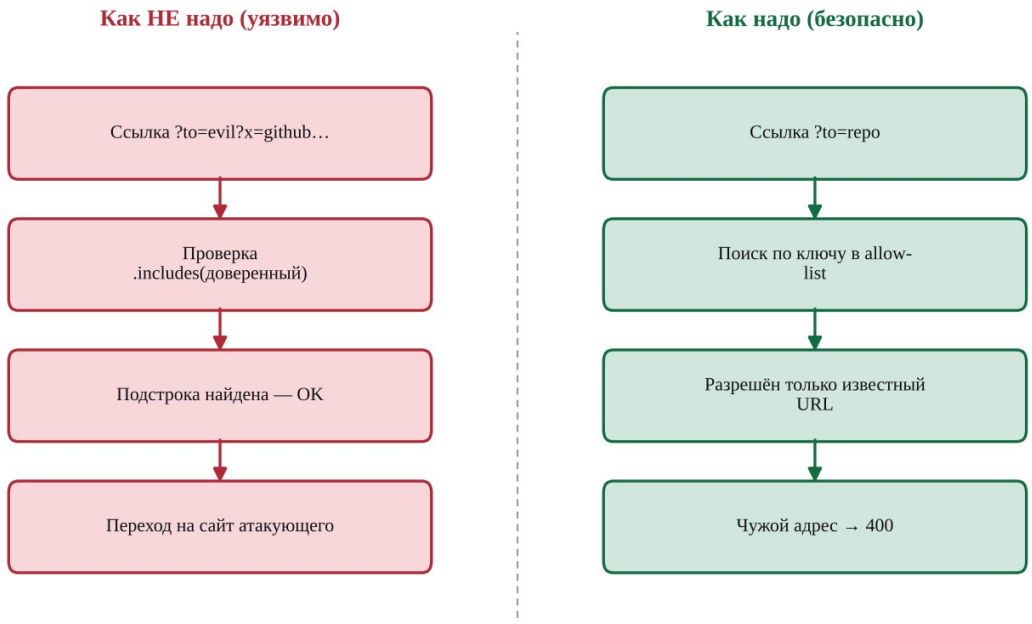


Рисунок 2 — Уязвимый и безопасный поток обработки (Открытое перенаправление (Open Redirect))

Было (уязвимо):

```
// УЯЗВИМО: проверка подстроки
const ALLOW = ['github.com/juice-shop/juice-shop']
app.get('/redirect', (req, res) => {
  const to = req.query.to
  if (ALLOW.some(a => to.includes(a)))   // легко обмануть вложением
    res.redirect(to)
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: allow-list точных целей / относительные пути
const ALLOW = new Map([
  ['repo', 'https://github.com/juice-shop/juice-shop']
])
app.get('/redirect', (req, res) => {
  const target = ALLOW.get(String(req.query.to))  // по ключу, не по URL
  if (!target) return res.status(400).end()
  res.redirect(target)
})
// либо только относительный путь: if (!to.startsWith('/') ||
to.startsWith('//')) reject
```

Уязвимость №3. Небезопасная конфигурация CORS

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Высокая.

3.3.1. Как нашли

Ответы API возвращают заголовок Access-Control-Allow-Origin, отражающий произвольный Origin запроса, вместе с Access-Control-Allow-Credentials: true. Это позволяет любому стороннему сайту читать ответы API от имени аутентифицированного пользователя.

3.3.2. Как использовать

С вредоносного домена выполняется кросс-доменный запрос с передачей кук/токена:

fetch('http://<IP>:3000/rest/user/whoami', { credentials: 'include' })

.then(r => r.json()).then(d => exfiltrate(d))

Браузер разрешает чтение ответа, так как сервер отражает Origin и разрешает credentials; данные жертвы утекают на сайт атакующего.

3.3.3. Как исправить

Рекомендуемая модель: Строгий allow-list Origin; не сочетать '*' с credentials.

Отражение произвольного Origin вместе с Allow-Credentials:true позволяет любому сайту читать ответы API жертвы. Модель: сверять Origin с белым списком и выставлять Access-Control-Allow-Origin только для разрешённых доменов; при использовании credentials запрещено значение '*'; по умолчанию CORS закрыт.

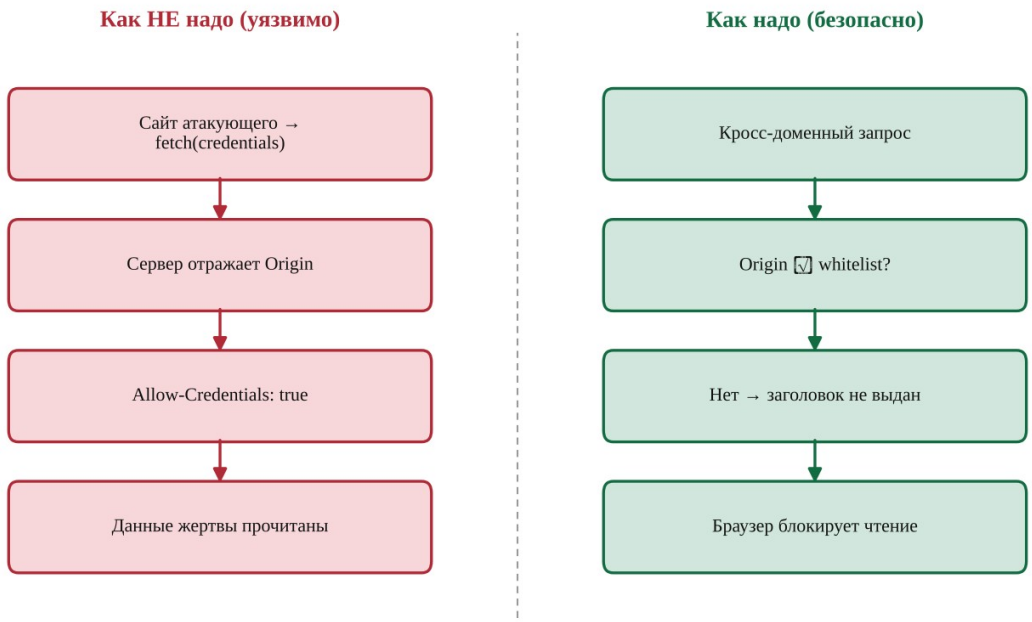


Рисунок 3 — Уязвимый и безопасный поток обработки (Небезопасная конфигурация CORS)

Было (уязвимо):

```
// УЯЗВИМО: Origin отражается, credentials разрешены
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', req.headers.origin) // любой
  res.header('Access-Control-Allow-Credentials', 'true')
  next()
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: allow-list доменов
import cors from 'cors'
const WHITELIST = ['https://shop.example.com']
app.use(cors({
  origin: (origin, cb) => cb(null, !origin || WHITELIST.includes(origin)),
  credentials: true   // '*' с credentials недопустимо
}))
```

Уязвимость №4. Загрязнение параметров HTTP (HTTP Parameter Pollution)

Категория OWASP: A03:2021 – Injection. **Критичность:** Средняя.

3.4.1. Как нашли

Сервер и промежуточные слои по-разному трактуют повторяющиеся одноимённые параметры (берут первый, последний или массив). Это рассогласование позволяет обойти проверки или изменить логику обработки заказа/referral-параметров.

3.4.2. Как использовать

В запрос добавляется дублирующийся параметр, чтобы проверочный слой увидел одно значение, а обработчик — другое:

```
POST /api/Feedbacks?userId=1&userId=2
```

Один компонент валидирует userId=1, а бизнес-логика использует userId=2, что приводит к подмене привязки записи.

3.4.3. Как исправить

Рекомендуемая модель: Явная нормализация параметров + строгая схема валидации.

HPP эксплуатирует рассогласование слоёв в трактовке повторяющихся параметров. Модель: приводить каждый параметр к одному ожидаемому типу (брать строго первое значение или запрещать массивы), валидировать по схеме, а идентификаторы субъекта брать из токена, а не из query.

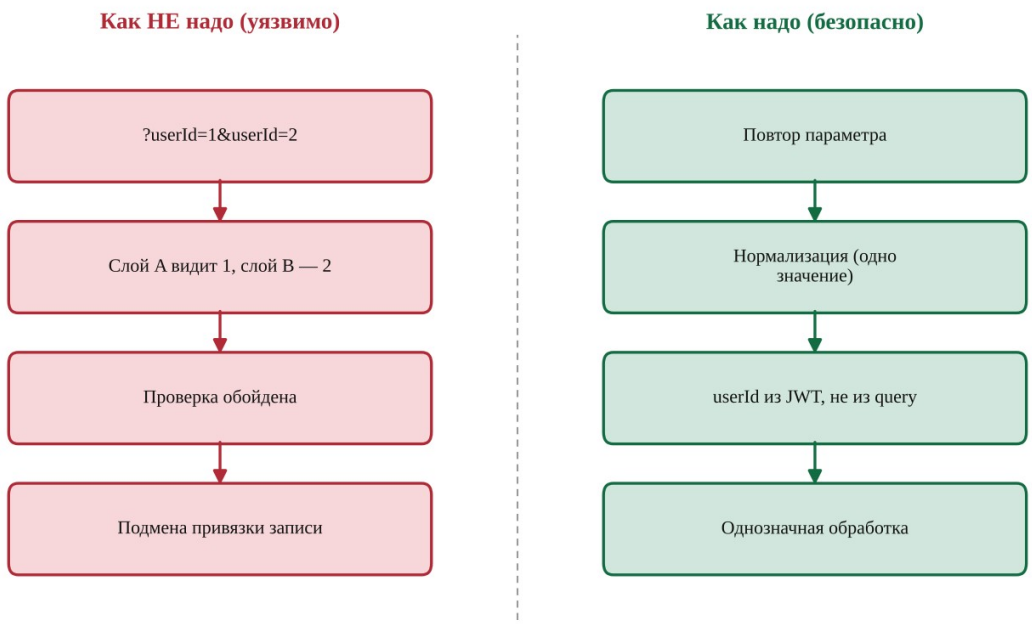


Рисунок 4 — Уязвимый и безопасный поток обработки (Загрязнение параметров HTTP (HTTP Parameter Pollution))

Было (уязвимо):

```
// УЯЗВИМО: userId приходит массивом при дублировании
// ?userId=1&userId=2  → req.query.userId = ['1','2']
const userId = req.query.userId   // неоднозначно для разных слоёв
saveFeedback({ userId, ... })
```

Стало (безопасно):

```
// БЕЗОПАСНО: нормализация + источник из токена
function one(v) { return Array.isArray(v) ? v[0] : v }   // строго одно
значение
const userId = req.user.id   // привязка к аутентифицированному
пользователю
if (!Number.isInteger(userId)) return res.status(400).end()
saveFeedback({ userId, ... })
```

Уязвимость №5. Обход клиентской защиты от XSS

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.5.1. Как нашли

Поле (например ник/комментарий) очищается только на клиенте перед отправкой. Отправив запрос напрямую в REST API (минуя Angular-форму), клиентский фильтр можно полностью обойти — сервер сохраняет нагрузку как есть.

3.5.2. Как использовать

Запрос формируется напрямую к API, минуя клиентскую очистку:

```
POST /api/Users  { "username": "<iframe
src=\"javascript:alert(`xss`)\">", ... }
```

Нагрузка сохраняется и срабатывает при последующем отображении, так как единственная защита была на стороне клиента.

3.5.3. Как исправить

Рекомендуемая модель: Серверная санитизация/экранирование (клиентская — не граница безопасности).

Единственная защита была на клиенте и обходится прямым запросом в API. Модель: любую очистку и экранирование выполнять на сервере при сохранении и/или при выводе; клиентскую фильтрацию считать лишь UX, а не защитой. Вывод — через авто-экранирование фреймворка, при необходимости HTML — через DOMPurify.

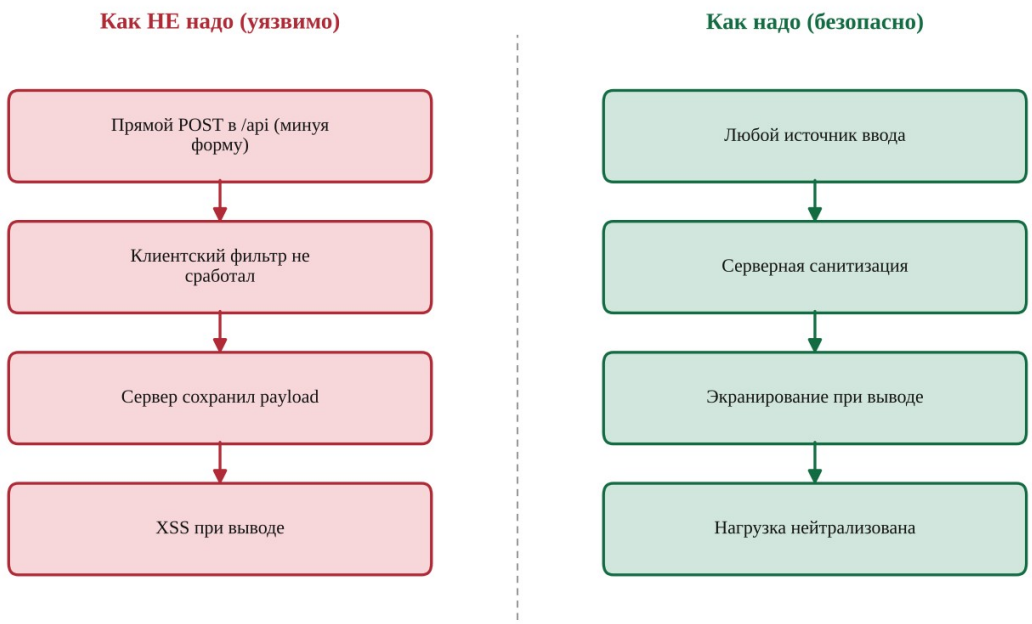


Рисунок 5 — Уязвимый и безопасный поток обработки (Обход клиентской защиты от XSS)

Было (уязвимо):

```
// УЯЗВИМО: чистка только на клиенте, сервер доверяет
// Angular: value = sanitizeOnClient(input)  // обходится прямым POST в
API
app.post('/api/Users', (req, res) => models.User.create(req.body))
```

Стало (безопасно):

```
// БЕЗОПАСНО: серверная санитизация + экранирование при выводе
import sanitizeHtml from 'sanitize-html'
app.post('/api/Users', (req, res) => {
  const username = sanitizeHtml(String(req.body.username), {
    allowedTags: [], allowedAttributes: {}   // только текст
  })
  models.User.create({ ...req.body, username })
})
// вывод: Angular {{ }} экранирует сам
```

Уязвимость №6. Обход серверной защиты от XSS

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.6.1. Как нашли

Сервер пытается чистить ввод по чёрному списку тегов/атрибутов (например удаляет <script>), но реализация неполна и обходится конструкциями, не попадающими под фильтр (iframe с javascript:, обработчики событий).

3.6.2. Как использовать

Подбирается payload, который серверный фильтр не распознаёт:

```
<iframe src="javascript:alert(`xss`)">
```

Тег не входит в чёрный список, проходит очистку и сохраняется, после чего исполняется у просматривающего пользователя.

3.6.3. Как исправить

Рекомендуемая модель: Allow-list санитайзер (DOMPurify/sanitize-html) вместо чёрного списка.

Чёрный список (удаление <script>) неполон и обходится. Модель: применять проверенный санитайзер по принципу белого списка допустимых тегов/атрибутов, а по умолчанию — экранировать вывод целиком; не изобретать собственные фильтры.

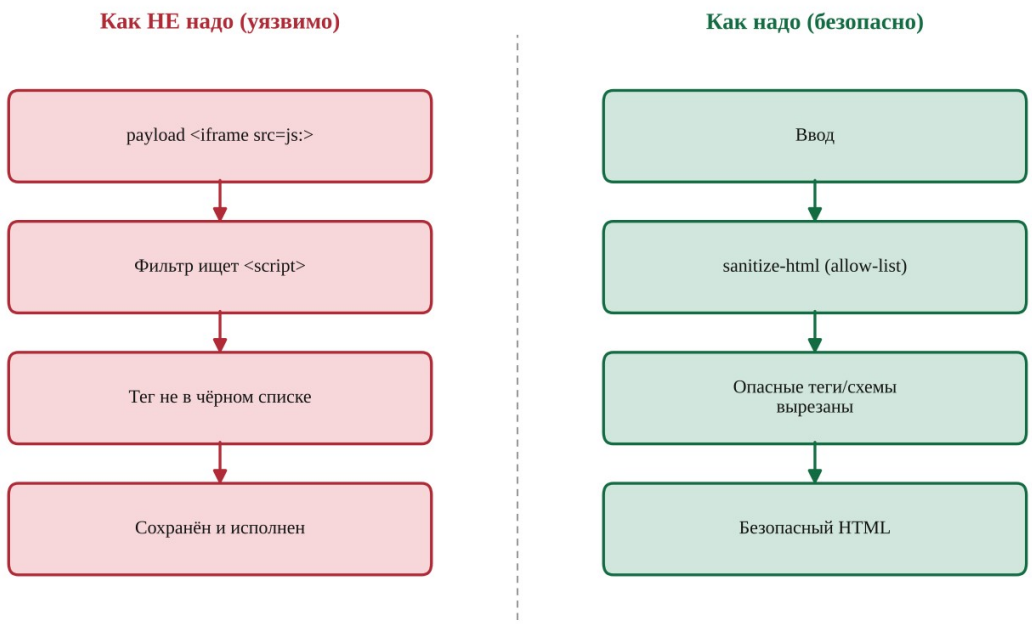


Рисунок 6 — Уязвимый и безопасный поток обработки (Обход серверной защиты от XSS)

Было (уязвимо):

```
// УЯЗВИМО: самодельный blacklist
const clean = input.replace(/<script>/gi, '')  // <iframe src=js:> пройдёт
```

Стало (безопасно):

```
// БЕЗОПАСНО: allow-list санитайзер
import sanitizeHtml from 'sanitize-html'
const clean = sanitizeHtml(input, {
  allowedTags: ['b', 'i', 'em', 'strong', 'a'],
  allowedAttributes: { a: ['href'] },
  allowedSchemes: ['http', 'https', 'mailto']   // javascript: запрещён
})
```

Уязвимость №7. Хранимый XSS только через API (без фронтенда)

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.7.1. Как нашли

Некоторые поля (например заголовок товара) недоступны для прямого ввода через интерфейс, но изменяемы через REST API. Сервер не валидирует и не экранирует их, что позволяет внедрить хранимую нагрузку чисто через API.

3.7.2. Как использовать

Через API изменяется поле, выводимое другим пользователям:

```
PUT /api/Products/1  { "description": "<iframe
src=\"javascript:alert(`xss`)\">" }
```

При открытии карточки товара нагрузка исполняется. Атака не требует фронтенда и обходит все клиентские проверки.

3.7.3. Как исправить

Рекомендуемая модель: Серверная валидация и экранирование всех полей, независимо от UI.

Поля, недоступные в UI, всё равно изменяемы через API и не проверяются. Модель: валидировать и экранировать каждое поле на сервере вне зависимости от того, доступно ли оно во фронтенде; выводить через авто-экранирование; закрыть изменение служебных полей авторизацией.

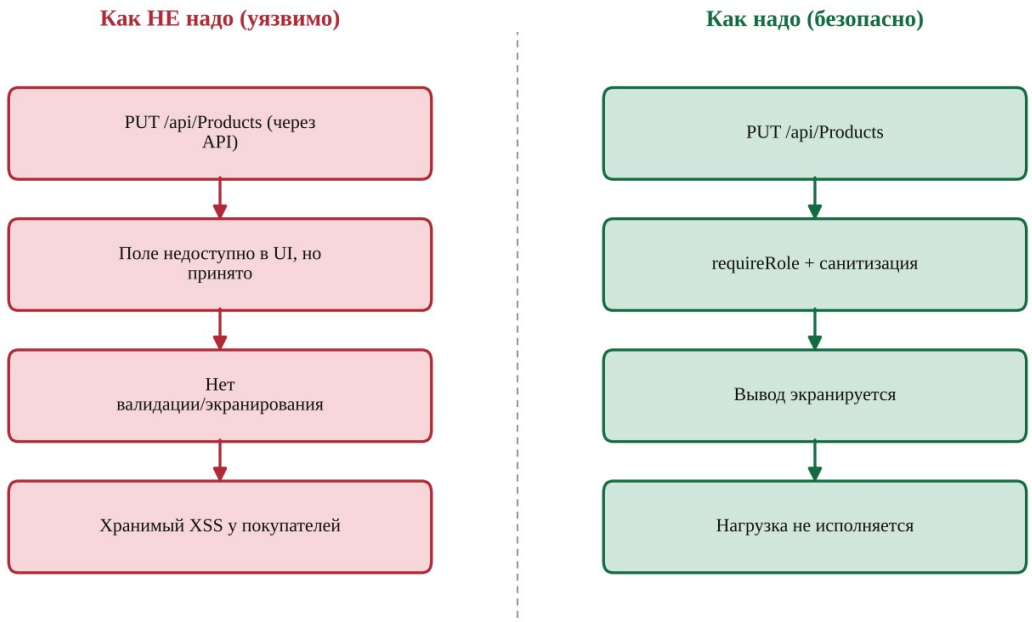


Рисунок 7 — Уязвимый и безопасный поток обработки (Хранимый XSS только через API (без фронтенда))

Было (уязвимо):

```
// УЯЗВИМО: поле меняется через API без проверки
```

```
app.put('/api/Products/:id', (req, res) =>
  models.Product.update(req.body, { where: { id: req.params.id } }))
```

Стало (безопасно):

```
// БЕЗОПАСНО: авторизация + валидация + экранирование
app.put('/api/Products/:id', requireRole('admin'), (req, res) => {
  const description = sanitizeHtml(String(req.body.description), {
    allowedTags: [], allowedAttributes: {} })
  models.Product.update({ description },
    { where: { id: Number(req.params.id) }, fields: ['description'] })
})
```

Уязвимость №8. Бизнес-логика: отрицательное количество в корзине

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Высокая.

3.8.1. Как нашли

Количество товара в корзине (/api/BasketItems) принимается без проверки знака и типа. Отрицательное или дробное значение проходит на сервер и влияет на итоговую сумму — классический дефект бизнес-логики.

3.8.2. Как использовать

Через перехват запроса в корзину выставляется отрицательное количество:

```
PUT /api/BasketItems/1  { "quantity": -50 }
```

Итоговая сумма уменьшается (вплоть до отрицательной), позволяя оплатить заказ значительно дешевле или получить «возврат».

3.8.3. Как исправить

Рекомендуемая модель: Серверная валидация диапазона (quantity — целое ≥ 1) + пересчёт суммы на сервере.

Отрицательное/дробное количество принимается и влияет на сумму. Модель: валидировать количество на сервере (целое, ≥1, с разумным

максимумом), итоговую стоимость всегда считать на сервере по каталожным ценам, никогда не доверяя клиентским суммам.

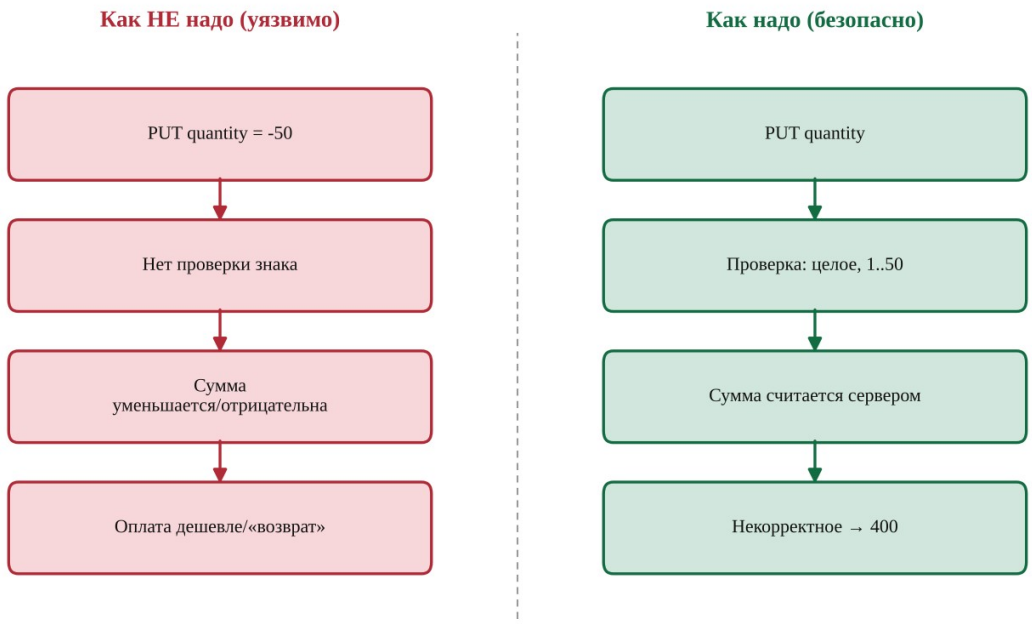


Рисунок 8 — Уязвимый и безопасный поток обработки (Бизнес-логика: отрицательное количество в корзине)

Было (уязвимо):

```
// УЯЗВИМО: количество без проверки знака/типа
app.put('/api/BasketItems/:id', (req, res) =>
  models.BasketItem.update({ quantity: req.body.quantity },
    { where: { id: req.params.id } }))
```

Стало (безопасно):

```
// БЕЗОПАСНО: валидация диапазона
app.put('/api/BasketItems/:id', async (req, res) => {
  const qty = Number(req.body.quantity)
  if (!Number.isInteger(qty) || qty < 1 || qty > 50)
    return res.status(400).json({ error: 'Invalid quantity' })
  await models.BasketItem.update({ quantity: qty },
    { where: { id: Number(req.params.id) } })
  // сумма пересчитывается сервером по ценам из БД
})
```

Уязвимость №9. Бизнес-логика: повторное использование купона / реферальный фрод

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Средняя.

3.9.1. Как нашли

Применение купона не фиксирует факт его использования на сервере: один и тот же код можно применить многократно (в т.ч. гонкой запросов), получая накопительную скидку. Нет серверного счётчика использований.

3.9.2. Как использовать

Один валидный купон применяется повторно, в т.ч. параллельными запросами (race condition):

for i in $(seq 1 20); do curl -X PUT .../rest/basket/1/coupon/<CODE> & done

Из-за отсутствия атомарной проверки скидка применяется многократно.

3.9.3. Как исправить

Рекомендуемая модель: Серверный учёт использований + атомарная проверка (защита от гонок).

Купон можно применить многократно, в т.ч. параллельными запросами. Модель: хранить факт и число использований на сервере, проверять и инкрементировать атомарно (транзакция/условный апдейт), ограничивать купон одним применением на заказ/пользователя.

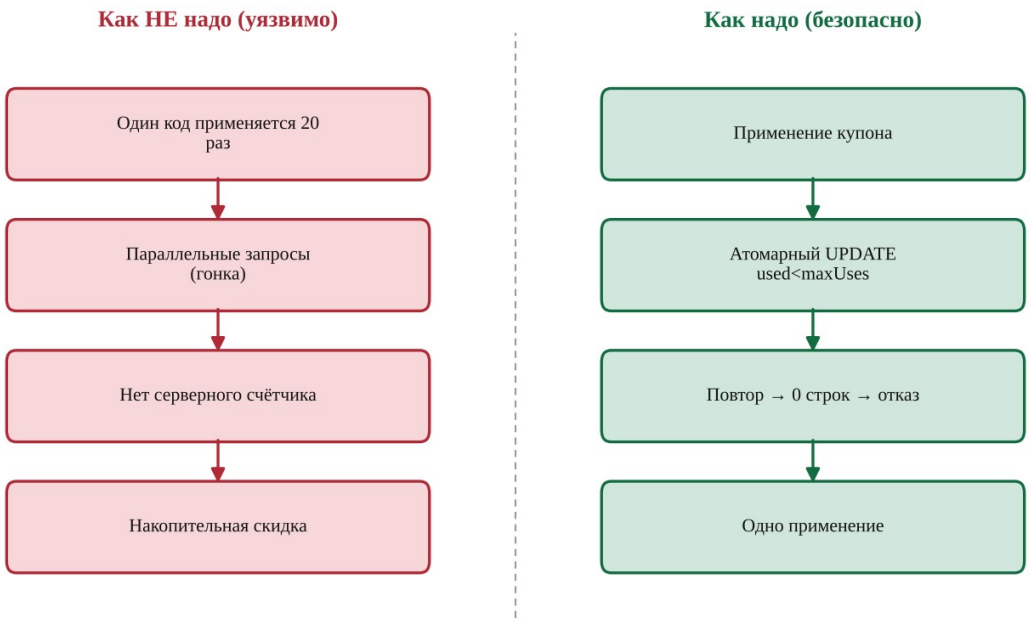


Рисунок 9 — Уязвимый и безопасный поток обработки (Бизнес-логика: повторное использование купона / реферальный фрод)

Было (уязвимо):

```
// УЯЗВИМО: нет учёта использований
app.put('/rest/basket/:id/coupon/:code', (req, res) => {
  const c = findCoupon(req.params.code)
  if (c) applyDiscount(req.params.id, c.discount) // можно повторять
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: атомарный расход использования
app.put('/rest/basket/:id/coupon/:code', async (req, res) => {
  // условный апдейт: списываем одно использование атомарно
  const [n] = await db.query(
    'UPDATE coupons SET used = used + 1 WHERE code = $c AND used <
maxUses',
    { bind: { c: req.params.code } })
  if (n === 0) return res.status(400).json({ error: 'Coupon not
applicable' })
  // привязка купона к корзине (один раз)
})
```

Уязвимость №10. Бизнес-логика: манипуляция балансом кошелька

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Критическая.

3.10.1. Как нашли

Пополнение внутреннего кошелька (/rest/wallet/balance) принимает сумму из тела запроса и начисляет её без фактической оплаты/сверки. Сервер доверяет значению amount, присланному клиентом.

3.10.2. Как использовать

В запросе на пополнение подставляется произвольная сумма:

```
PUT /rest/wallet/balance  { "balance": 1000000, "paymentId": "" }
```

Баланс начисляется без реальной оплаты, что равносильно краже средств магазина.

3.10.3. Как исправить

Рекомендуемая модель: Сервер проводит платёж и сверяет его с провайдером; клиент не задаёт сумму.

Баланс начисляется по сумме из запроса без фактической оплаты. Модель: пополнение проводить только через серверную инициацию платежа у провайдера и подтверждение webhook'ом; начислять ровно подтверждённую провайдером сумму; клиентские значения суммы игнорировать.

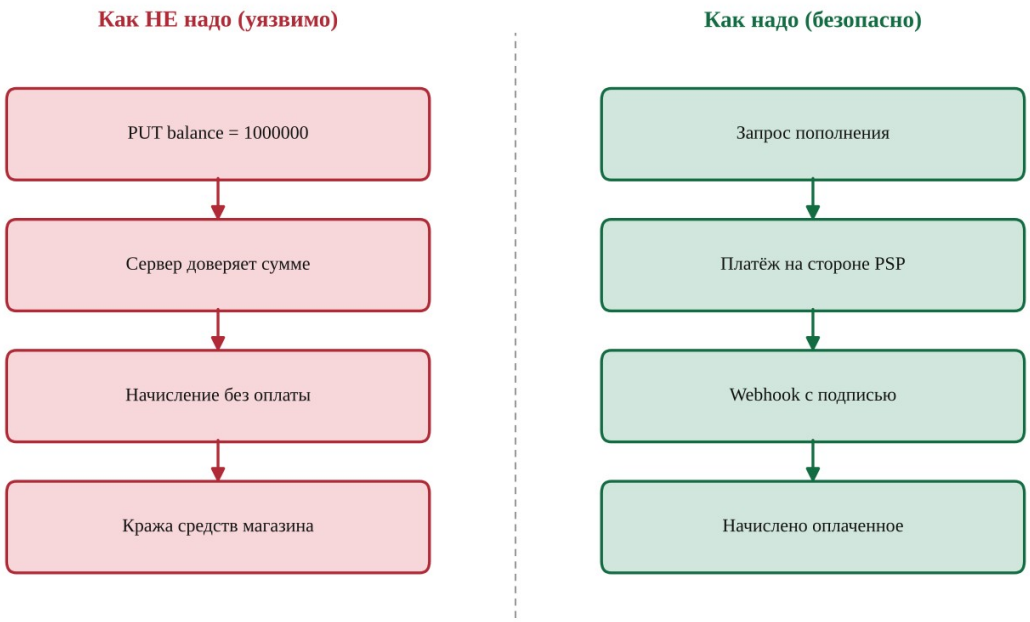


Рисунок 10 — Уязвимый и безопасный поток обработки (Бизнес-логика: манипуляция балансом кошелька)

Было (уязвимо):

```
// УЯЗВИМО: сумма из запроса начисляется напрямую
app.put('/rest/wallet/balance', (req, res) => {
  wallet.add(req.user.id, req.body.balance)  // без оплаты
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: начисление только по подтверждённому платежу
app.post('/rest/wallet/topup', async (req, res) => {
  const intent = await psp.createPaymentIntent(req.user.id,
req.body.amount)
  res.json({ clientSecret: intent.clientSecret })  // оплата на стороне
PSP
})
// webhook от платёжного провайдера (подпись проверяется):
app.post('/webhook/psp', verifyPspSignature, async (req, res) => {
  const ev = req.body
  if (ev.type === 'payment.succeeded')
    await wallet.add(ev.userId, ev.amountConfirmed)  // ровно оплаченное
})
```

Уязвимость №11. Получение Deluxe-членства без оплаты

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.11.1. Как нашли

Переход на Deluxe-членство (/rest/deluxe-membership) проверяет факт оплаты недостаточно строго: статус платежа можно подменить или вызвать апгрейд напрямую, минуя оплату.

3.11.2. Как использовать

Напрямую вызывается эндпоинт апгрейда с подменой способа оплаты:

```
POST /rest/deluxe-membership  { "paymentMode": "wallet" }
```

при нулевом балансе/без реального списания членство активируется.

3.11.3. Как исправить

Рекомендуемая модель: Серверная проверка факта оплаты перед апгрейдом роли.

Статус оплаты проверяется слабо/на клиенте. Модель: апгрейд членства выполнять только после серверного подтверждения успешного платежа (по его идентификатору у провайдера), а не по значению из запроса.

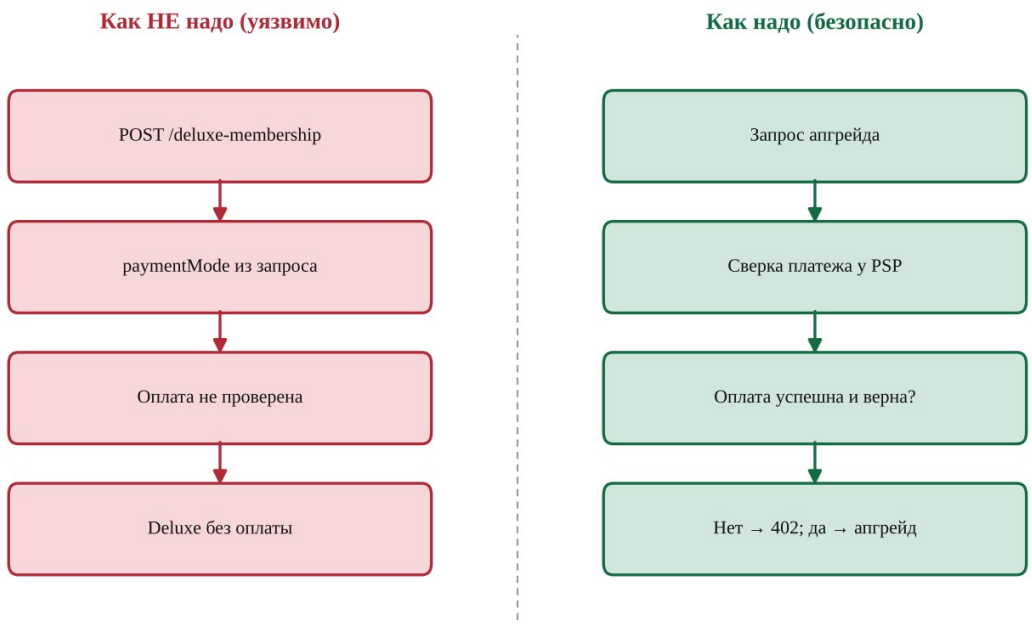


Рисунок 11 — Уязвимый и безопасный поток обработки (Получение Deluxe-членства без оплаты)

Было (уязвимо):

```
// УЯЗВИМО: апгрейд без подтверждения оплаты
app.post('/rest/deluxe-membership', (req, res) => {
  upgradeToDeluxe(req.user.id)   // paymentMode из запроса не сверяется
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: сверка оплаты у провайдера
app.post('/rest/deluxe-membership', async (req, res) => {
  const pay = await psp.getPayment(req.body.paymentId)
  if (!pay || pay.status !== 'succeeded' || pay.userId !== req.user.id
      || pay.amount < DELUXE_PRICE)
    return res.status(402).json({ error: 'Payment required' })
  await upgradeToDeluxe(req.user.id)
})
```

Уязвимость №12. Перечисление пользователей (user enumeration)

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Средняя.

3.12.1. Как нашли

Ответы форм входа и восстановления различаются для существующих и несуществующих email (текст ошибки, код ответа, время отклика). Это позволяет собрать список валидных учётных записей.

3.12.2. Как использовать

Перебором по списку email сравниваются ответы:

```
POST /rest/user/forgot-password  { "email": "test@x.op" }
```

Существующий аккаунт возвращает секретный вопрос, несуществующий — ошибку; по различию составляется список валидных пользователей для дальнейших атак.

3.12.3. Как исправить

Рекомендуемая модель: Единообразные ответы и тайминг для существующих и несуществующих аккаунтов.

Различия в ответах раскрывают валидные email. Модель: при входе и восстановлении возвращать одинаковый нейтральный ответ независимо от существования аккаунта; выравнивать время отклика; ограничивать частоту, чтобы затруднить массовое перечисление.

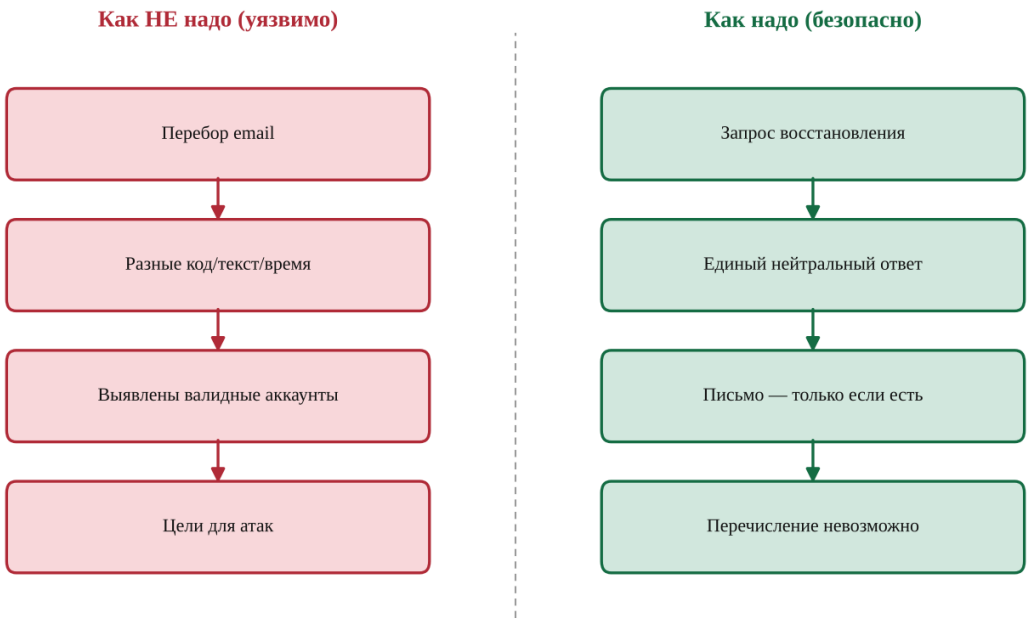


Рисунок 12 — Уязвимый и безопасный поток обработки (Перечисление пользователей (user enumeration))

Было (уязвимо):

```
// УЯЗВИМО: разные ответы
if (!user) return res.status(404).json({ error: 'User not found' })
return res.json({ securityQuestion: user.question })
```

Стало (безопасно):

```
// БЕЗОПАСНО: единообразный ответ
// всегда одинаковый ответ, письмо (если аккаунт есть) уходит асинхронно
await maybeSendResetEmail(email)   // внутри: есть пользователь — шлём, нет — нет
return res.json({ message: 'Если аккаунт существует, мы отправили инструкции' })
// + rate-limit, + выравнивание времени ответа
```

Уязвимость №13. JWT без срока действия (токен не истекает)

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Высокая.

3.13.1. Как нашли

Выдаваемый JWT не содержит (или игнорирует) claim exp и не отзывается на сервере при выходе. На jwt.io видно отсутствие срока действия: однажды полученный токен действует бессрочно.

3.13.2. Как использовать

Перехваченный (например через XSS/логи) токен используется спустя длительное время:

```
curl .../rest/user/whoami -H 'Authorization: Bearer <старый_JWT>'
```

Сервер принимает токен, так как срок не проверяется и выход не инвалидирует сессию.

3.13.3. Как исправить

Рекомендуемая модель: Короткий срок жизни (exp) + refresh-токены + серверная ревокация.

Токен без exp и без инвалидации действует вечно. Модель: задавать короткий срок жизни access-токена (минуты), использовать refresh-токены, вести серверный список отзыва (denylist) и инвалидировать сессии при выходе/смене пароля.

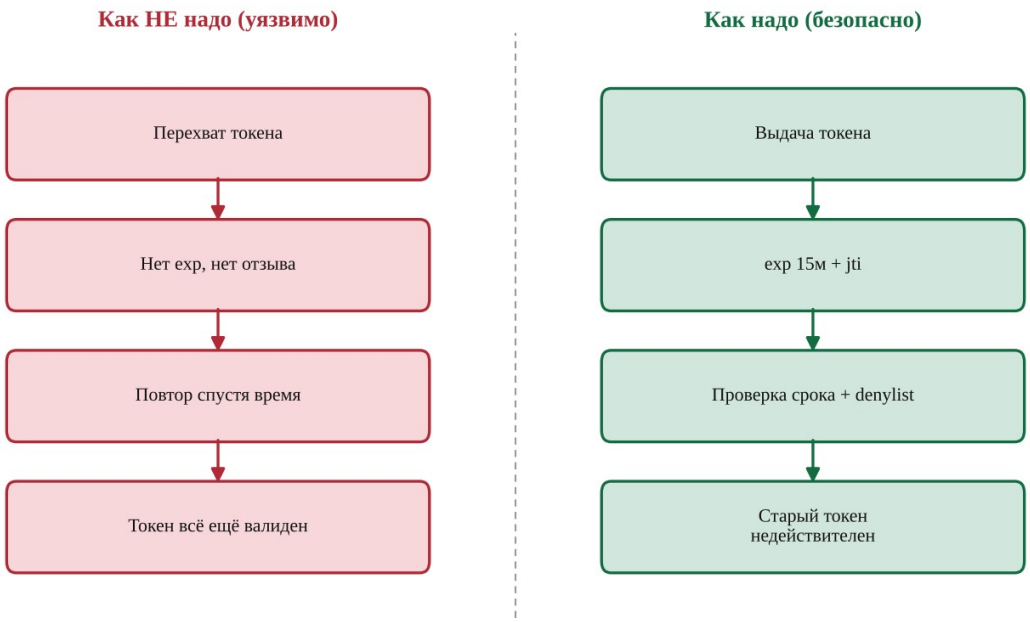


Рисунок 13 — Уязвимый и безопасный поток обработки (JWT без срока действия (токен не истекает))

Было (уязвимо):

```
// УЯЗВИМО: токен без срока
const token = jwt.sign({ sub: user.id }, key)  // нет expiresIn
jwt.verify(token, key)                          // срок не проверяется
```

Стало (безопасно):

```
// БЕЗОПАСНО: срок + проверка + ревокация
const token = jwt.sign({ sub: user.id }, key, {
  algorithm: 'RS256', expiresIn: '15m', jwtid: uuid()
})
jwt.verify(token, pub, { algorithms: ['RS256'], maxAge: '15m' })
// при выходе/смене пароля — jti в denylist; доступ продлевается refresh-токеном
```

Уязвимость №14. Предсказуемый пароль OAuth-аккаунта

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Высокая.

3.14.1. Как нашли

Для OAuth-входа приложение хранит «пароль», производный от email пользователя обратимым преобразованием (перевёрнутый email в base64). Зная email, пароль вычисляется без перебора.

3.14.2. Как использовать

Пароль восстанавливается по известному email:

echo -n 'po.hs-eciuj@nnvalleh' | base64 # перевёрнутый email → base64

Полученная строка используется как пароль для прямого входа в аккаунт жертвы.

3.14.3. Как исправить

Рекомендуемая модель: Криптослучайный пароль/без пароля для OAuth; отказ от выводимых секретов.

«Пароль» вычисляется из email обратимым кодированием. Модель: для OAuth не хранить производных от публичных данных паролей — либо вовсе не заводить локальный пароль (вход только через провайдера), либо генерировать криптослучайный секрет и хранить его хешем (argon2/bcrypt).

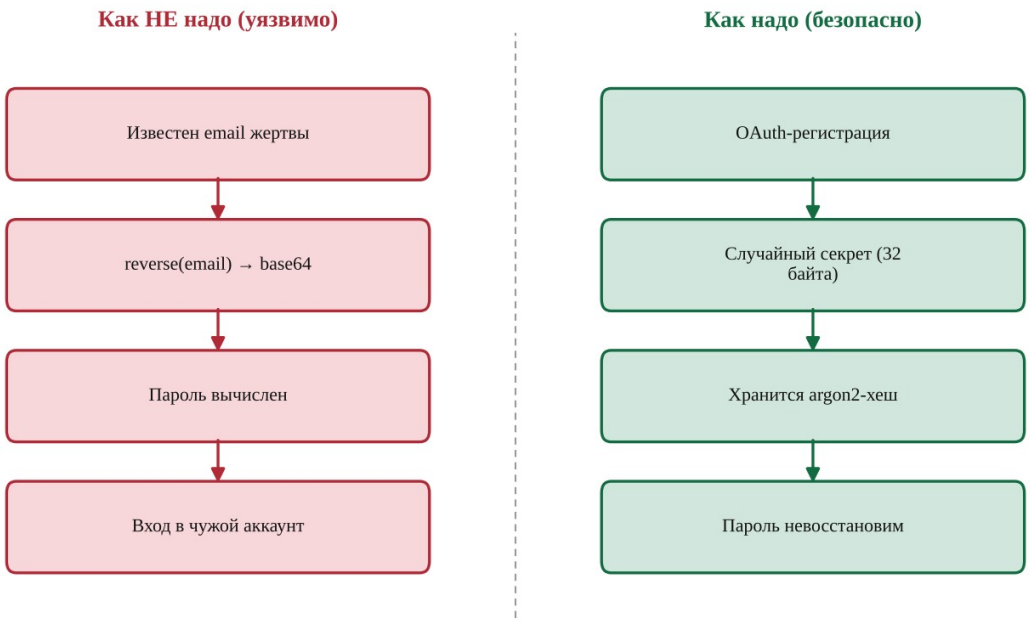


Рисунок 14 — Уязвимый и безопасный поток обработки (Предсказуемый пароль OAuth-аккаунта)

Было (уязвимо):

```
// УЯЗВИМО: пароль выводится из email
const password = Buffer.from(email.split('').reverse().join(''))
  .toString('base64')   // зная email, пароль вычислим
```

Стало (безопасно):

```
// БЕЗОПАСНО: случайный секрет + хеш, либо без локального пароля
import crypto from 'crypto'; import argon2 from 'argon2'
const random = crypto.randomBytes(32).toString('base64url')
const hash = await argon2.hash(random)  // хранится только хеш
// предпочтительно: для OAuth не заводить локальный пароль вовсе
```

Уязвимость №15. Нарушение приватности: доступ к данным и удалению чужого аккаунта

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.15.1. Как нашли

Операции выгрузки персональных данных и удаления аккаунта (GDPR) определяют субъекта по идентификатору из запроса, а не из токена. Это позволяет выгрузить или удалить данные другого пользователя.

3.15.2. Как использовать

В запросе экспорта/удаления подставляется чужой идентификатор:

```
POST /rest/user/data-export  { "userId": 2 }
```

Сервер возвращает или удаляет данные указанного пользователя без проверки принадлежности.

3.15.3. Как исправить

Рекомендуемая модель: Привязка операций с данными к субъекту из токена + подтверждение.

Экспорт/удаление определяют субъекта по идентификатору из запроса. Модель: выполнять операции только над данными текущего пользователя (личность — из JWT), требовать подтверждения (повторная аутентификация) для удаления и вести аудит.

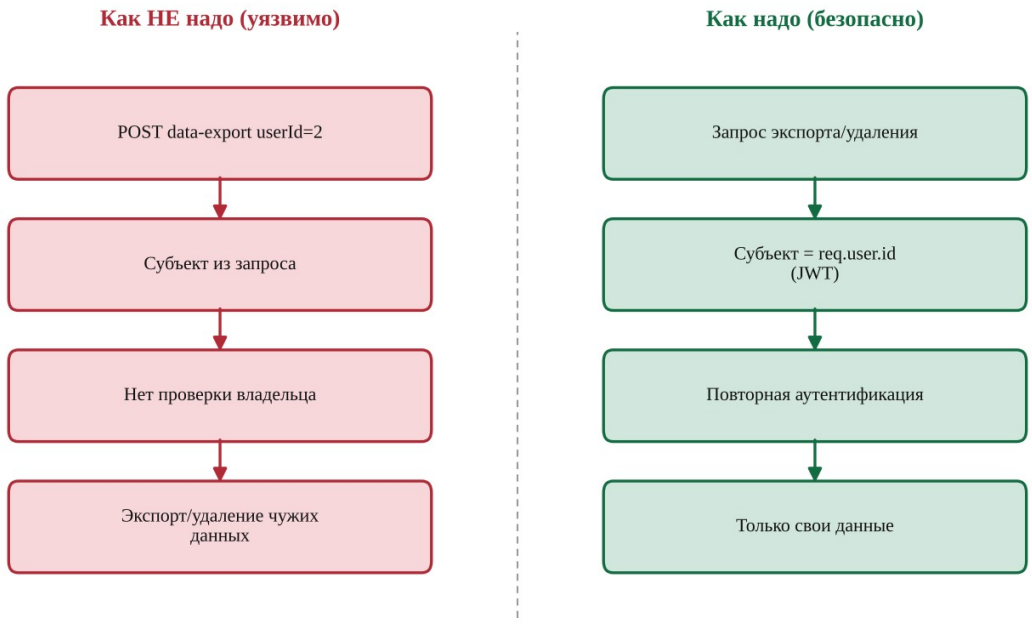


Рисунок 15 — Уязвимый и безопасный поток обработки (Нарушение приватности: доступ к данным и удалению чужого аккаунта)

Было (уязвимо):

```
// УЯЗВИМО: субъект из запроса
app.post('/rest/user/data-export', (req, res) =>
  res.json(exportUserData(req.body.userId)))   // любой userId
```

Стало (безопасно):

```
// БЕЗОПАСНО: только свои данные + подтверждение
app.post('/rest/user/data-export', async (req, res) => {
  const data = await exportUserData(req.user.id)  // из токена
  audit.log(req.user.id, 'export')
  res.json(data)
})
app.delete('/rest/user', reauth, async (req, res) => {
  await eraseUser(req.user.id); res.status(204).end()
})
```

Уязвимость №16. Раскрытие внутренних метрик (/metrics)

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Средняя.

3.16.1. Как нашли

Эндпоинт /metrics (формат Prometheus) доступен без аутентификации и раскрывает внутренние показатели: число запросов, ошибки, сведения о версии и окружении, пригодные для разведки.

3.16.2. Как использовать

Открывается http://<IP>:3000/metrics — отдаётся дамп метрик с технической информацией о приложении и инфраструктуре, помогающей планировать дальнейшие атаки.

3.16.3. Как исправить

Рекомендуемая модель: Ограничение доступа к служебным эндпоинтам (сеть/роль/отдельный порт).

Эндпоинт /metrics доступен публично и помогает разведке. Модель: закрывать служебные эндпоинты — выносить на внутренний интерфейс/порт, требовать аутентификацию или ограничивать по сети (только система мониторинга); не раскрывать детали окружения.

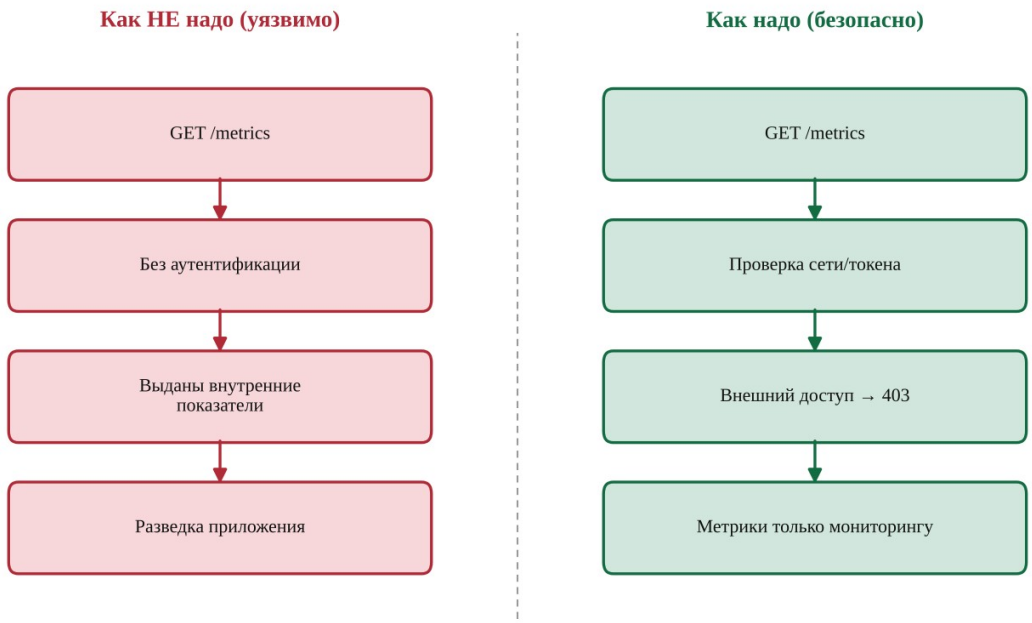


Рисунок 16 — Уязвимый и безопасный поток обработки (Раскрытие внутренних метрик (/metrics))

Было (уязвимо):

```
// УЯЗВИМО: метрики доступны всем
app.get('/metrics', (req, res) => res.send(promRegister.metrics()))
```

Стало (безопасно):

```
// БЕЗОПАСНО: доступ только из доверенной сети / по токену
app.get('/metrics', (req, res) => {
  if (!isFromMonitoringNet(req.ip) &&
      req.headers['x-metrics-token'] !== process.env.METRICS_TOKEN)
    return res.status(403).end()
  res.send(promRegister.metrics())
})
// лучше — отдельный внутренний порт, не доступный извне
```

Уязвимость №17. Устаревший B2B-интерфейс обработки XML

Категория OWASP: A06:2021 – Vulnerable and Outdated Components.

Критичность: Высокая.

3.17.1. Как нашли

В приложении присутствует устаревший («deprecated») B2B-интерфейс приёма заказов в XML, использующий небезопасный парсер. Он не выведен из эксплуатации и доступен, создавая поверхность для XML-атак (расширение сущностей, DoS).

3.17.2. Как использовать

В устаревший интерфейс отправляется XML с рекурсивным расширением сущностей (billion laughs):

```
<!DOCTYPE lolz [ <!ENTITY a "aaaa..."> <!ENTITY b
"&a;&a;&a;..."> ]><x>&b;</x>
```

Парсер исчерпывает память/CPU — отказ в обслуживании; при включённых внешних сущностях возможно чтение файлов.

3.17.3. Как исправить

Рекомендуемая модель: Вывод устаревшего интерфейса из эксплуатации + безопасный XML-парсинг.

Устаревший XML-интерфейс с опасным парсером остаётся доступным. Модель: отключить неиспользуемые/устаревшие интерфейсы; если приём XML необходим — запретить DTD и внешние сущности, ограничить глубину/размер документа и число расширений сущностей, предпочесть JSON.

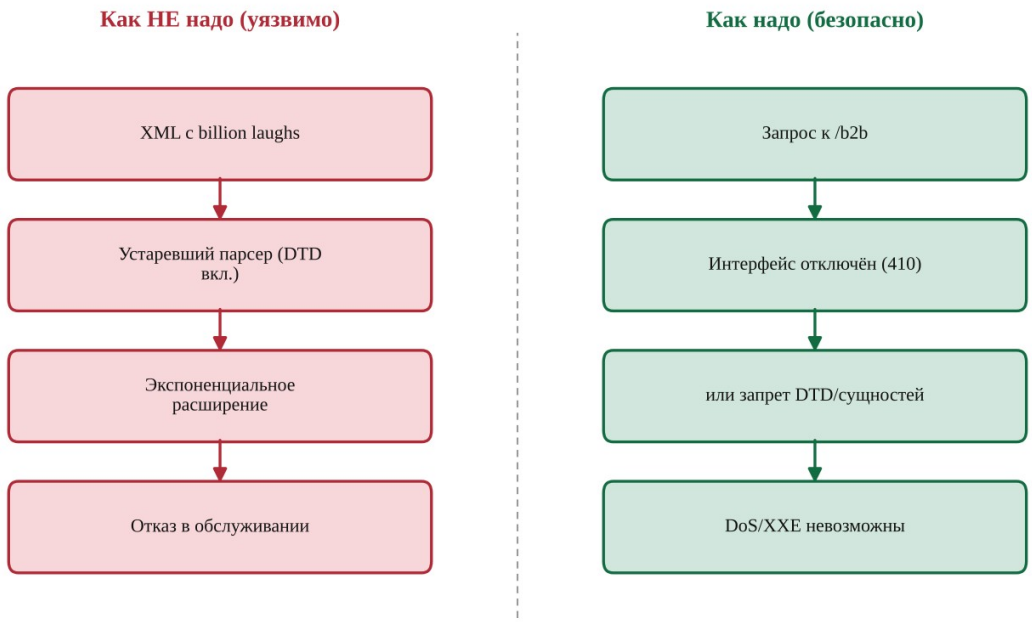


Рисунок 17 — Уязвимый и безопасный поток обработки (Устаревший B2B-интерфейс обработки XML)

Было (уязвимо):

```
// УЯЗВИМО: устаревший интерфейс, парсер с DTD/сущностями
app.post('/b2b/v1/orders', (req, res) => {
  const doc = libxml.parseXml(req.body, { noent: true, dtdload: true })
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: интерфейс отключён или парсинг обезврежен
// 1) если не нужен — удалить маршрут полностью (410 Gone)
app.post('/b2b/v1/orders', (req, res) => res.status(410).end())

// 2) если нужен — безопасный разбор:
if (/<!DOCTYPE/i.test(req.body)) return res.status(400).end()
const doc = libxml.parseXml(req.body, { noent:false, dtdload:false,
nonet:true })
```

Уязвимость №18. Чтение произвольных файлов (path traversal в API)

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Критическая.

3.18.1. Как нашли

Эндпоинт отдачи файла (например страницы/вложения) принимает имя/путь из запроса и формирует путь на диске без нормализации, что допускает выход за пределы каталога (path traversal) и чтение системных файлов.

3.18.2. Как использовать

В параметр файла подставляется обход каталога:

```
GET /ftp/..%2f..%2f..%2fetc%2fpasswd
```

Сервер отдаёт содержимое произвольного файла за пределами разрешённого каталога.

3.18.3. Как исправить

Рекомендуемая модель: Нормализация пути + canonical-проверка внутри базового каталога.

Путь берётся из запроса без нормализации, что допускает ../ и чтение системных файлов. Модель: декодировать и нормализовать путь, отклонять обходы, строить абсолютный путь и проверять, что он лежит строго внутри разрешённого каталога; отдавать только файлы из белого списка.

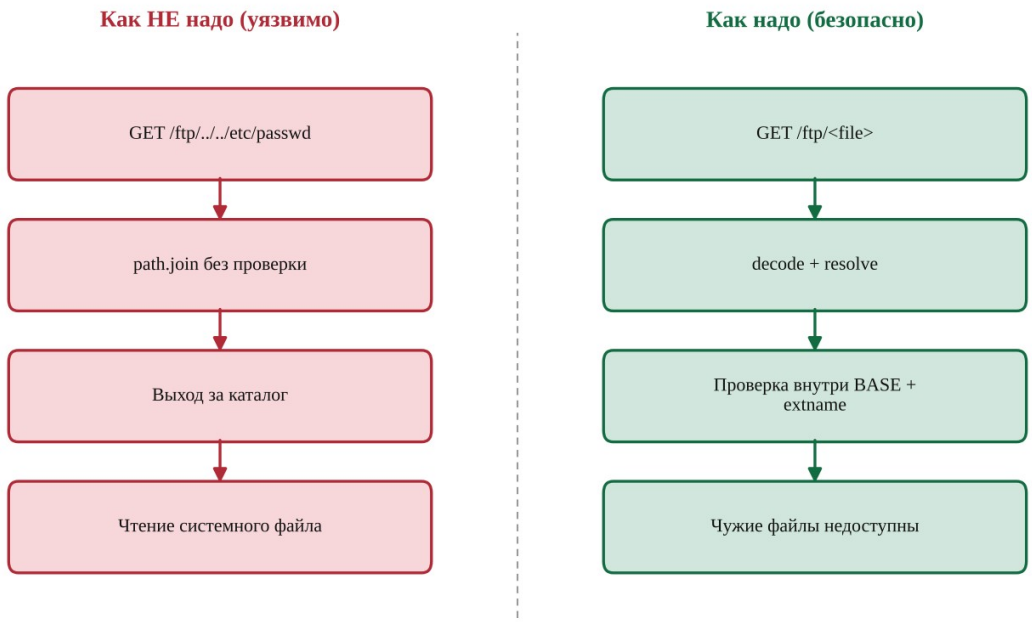


Рисунок 18 — Уязвимый и безопасный поток обработки (Чтение произвольных файлов (path traversal в API))

Было (уязвимо):

```
// УЯЗВИМО: путь из запроса
app.get('/ftp/*', (req, res) => {
  res.sendFile(path.join('/app/ftp', req.params[0]))  // ../ пройдёт
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: canonical-проверка внутри BASE
const BASE = path.resolve('/app/ftp')
app.get('/ftp/*', (req, res) => {
  const decoded = decodeURIComponent(req.params[0])
  const full = path.resolve(BASE, decoded)
  if (!full.startsWith(BASE + path.sep))   // строго внутри BASE
    return res.status(400).end()
  if (!['.md', '.pdf'].includes(path.extname(full)))
    return res.status(404).end()
  res.sendFile(full)
})
```

Уязвимость №19. Отказ в обслуживании через разбор YAML (YAML-бомба)

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Высокая.

3.19.1. Как нашли

Функция импорта конфигурации/заказа принимает YAML и разбирает его небезопасным парсером, раскрывающим якоря и алиасы без ограничений, что позволяет построить экспоненциально расширяющуюся структуру.

3.19.2. Как использовать

Загружается YAML с вложенными алиасами (аналог billion laughs):

a: &a ["x","x"]

b: &b [*a,*a]

c: &c [*b,*b]

... # экспоненциальный рост

Разбор исчерпывает память и CPU — приложение перестаёт отвечать (DoS).

3.19.3. Как исправить

Рекомендуемая модель: Безопасный парсер YAML (без раскрытия якорей) + лимиты и таймауты.

Разбор YAML с неограниченным раскрытием якорей/алиасов приводит к экспоненциальному росту и DoS. Модель: использовать безопасный загрузчик (FAILSAFE/CORE schema без мерджей и бесконтрольных алиасов), ограничивать размер входа, глубину и время разбора, выполнять его в изолированном воркере.

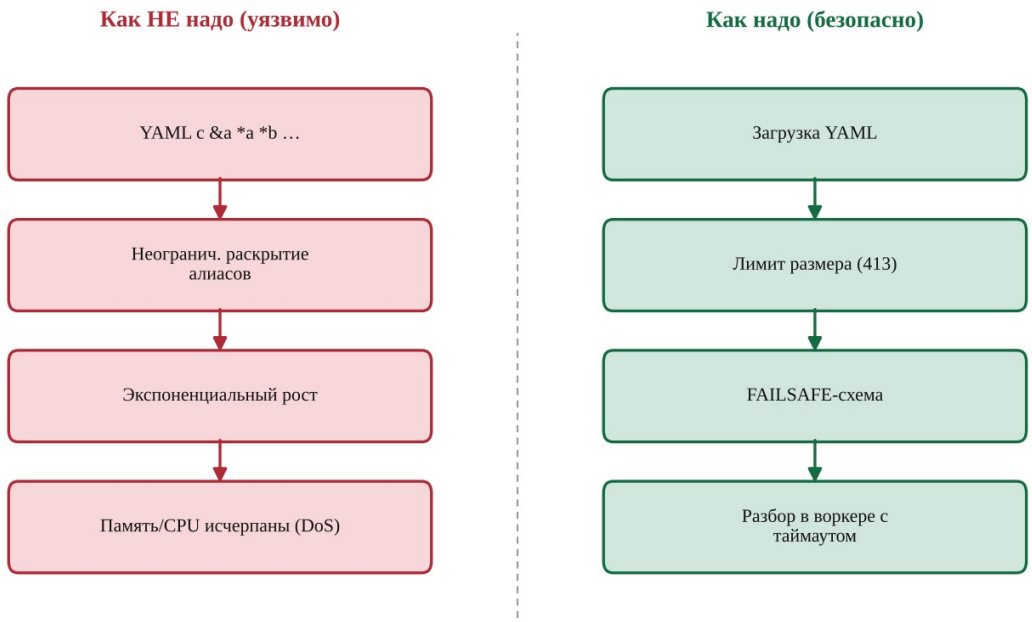


Рисунок 19 — Уязвимый и безопасный поток обработки (Отказ в обслуживании через разбор YAML (YAML-бомба))

Было (уязвимо):

```
// УЯЗВИМО: небезопасный разбор без лимитов
const yaml = require('js-yaml')
const data = yaml.load(req.body)   // якоря/алиасы раскрываются
неограниченно
```

Стало (безопасно):

```
// БЕЗОПАСНО: safe-схема + лимит размера + таймаут
const yaml = require('js-yaml')
if (req.body.length > 64 * 1024) return res.status(413).end() // лимит
размера
const data = yaml.load(req.body, { schema: yaml.FAILSAFE_SCHEMA })
// тяжёлый разбор — в воркере с таймаутом и ограничением памяти
```

Уязвимость №20. Смена пароля без текущего пароля

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Высокая.

3.20.1. Как нашли

Операция смены пароля не требует и не проверяет текущий пароль. Это облегчает захват аккаунта при перехвате сессии/токена и усиливает последствия CSRF.

3.20.2. Как использовать

Пароль меняется без знания текущего:

```
GET /rest/user/change-password?new=newpass&repeat=newpass
```

Поле current отсутствует/игнорируется, поэтому при наличии сессии пароль меняется напрямую.

3.20.3. Как исправить

Рекомендуемая модель: Обязательная проверка текущего пароля при смене.

Смена пароля не требует текущего, что облегчает захват при перехвате сессии и усиливает CSRF. Модель: всегда требовать и проверять текущий пароль (безопасным сравнением хешей) перед сменой; уведомлять пользователя и инвалидировать прочие сессии.

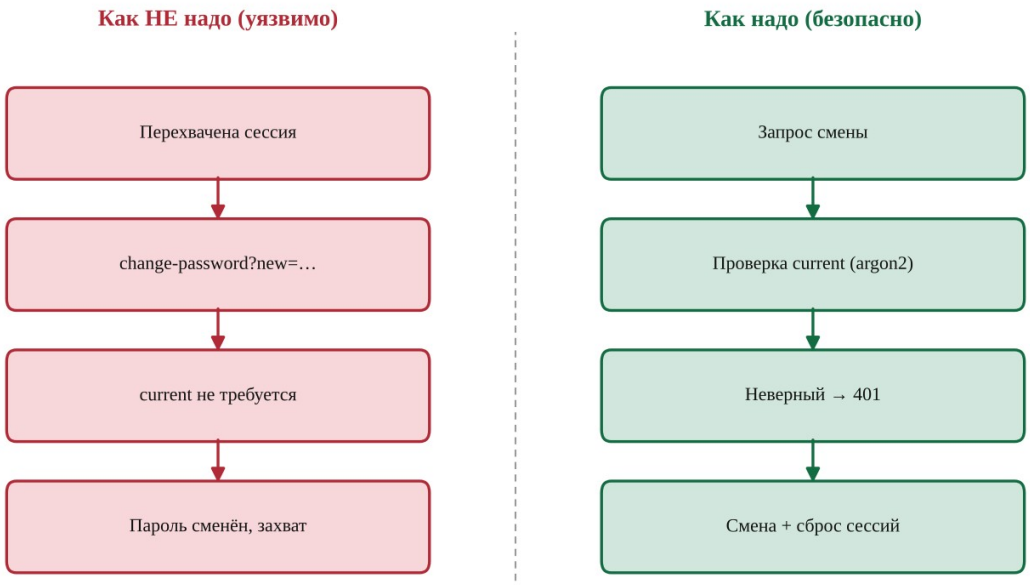


Рисунок 20 — Уязвимый и безопасный поток обработки (Смена пароля без текущего пароля)

Было (уязвимо):

```
// УЯЗВИМО: current не проверяется
app.get('/rest/user/change-password', (req, res) => {
  if (req.query.new === req.query.repeat)
    setPassword(req.user, req.query.new)
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: проверка текущего пароля (и POST)
app.post('/rest/user/change-password', async (req, res) => {
  const { current, new: n, repeat } = req.body
  if (n !== repeat) return res.status(400).end()
  if (!(await argon2.verify(req.user.password, current)))
    return res.status(401).json({ error: 'Wrong current password' })
  await setStrongPassword(req.user, n)
  await invalidateOtherSessions(req.user.id)
})
```

Уязвимость №21. Раскрытие информации через подробные сообщения об ошибках

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Средняя.

3.21.1. Как нашли

При ошибках сервер возвращает клиенту подробные стектрейсы, SQL-сообщения и пути файлов (например при кавычке в параметре). Это раскрывает внутреннее устройство приложения и облегчает другие атаки.

3.21.2. Как использовать

Некорректный ввод провоцирует исключение с детализацией:

```
GET /rest/products/search?q='
```

В ответе — текст SQL-запроса, версия ORM и путь файла, что ускоряет подготовку инъекций.

3.21.3. Как исправить

Рекомендуемая модель: Общие сообщения об ошибках для клиента + детальное серверное логирование.

Подробные стектрейсы и SQL-ошибки раскрывают внутреннее устройство. Модель: клиенту возвращать обобщённое сообщение и идентификатор инцидента, а детали писать в серверный лог; отключать режим разработки в продакшене; добавить централизованный обработчик ошибок.

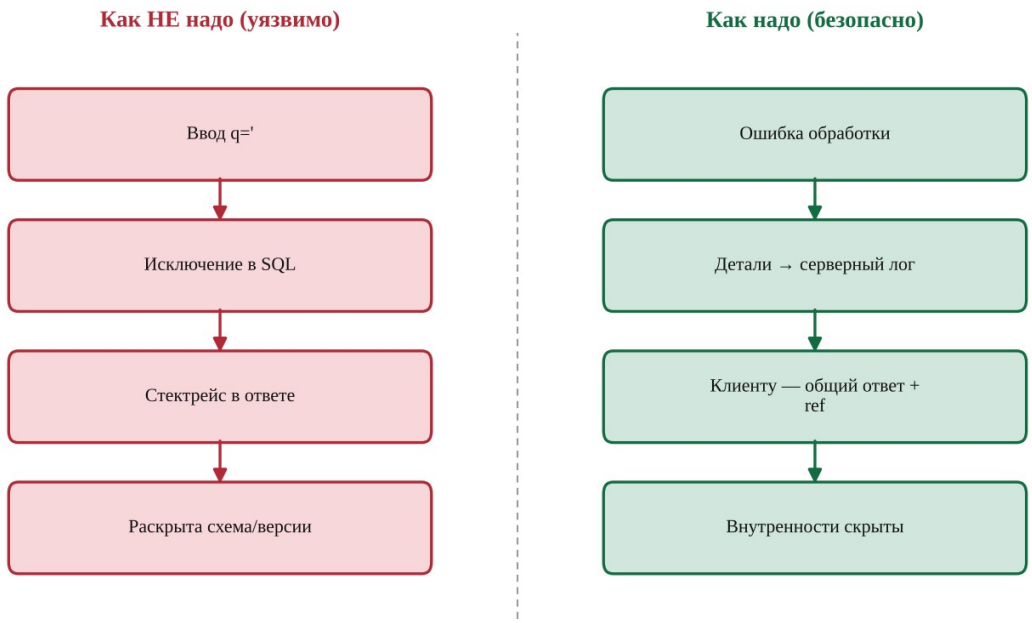


Рисунок 21 — Уязвимый и безопасный поток обработки (Раскрытие информации через подробные сообщения об ошибках)

Было (уязвимо):

```
// УЯЗВИМО: стектрейс уходит клиенту
app.use((err, req, res, next) => {
  res.status(500).send(err.stack)   // SQL, пути, версии
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: общий ответ + детали в лог
app.use((err, req, res, next) => {
  const id = uuid()
  logger.error({ id, err: err.stack })        // детали — только в лог
```

```
  res.status(500).json({ error: 'Internal error', ref: id })
})
// NODE_ENV=production; отключить подробные ошибки фреймворка
```

Уязвимость №22. Слабая политика паролей

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Средняя.

3.22.1. Как нашли

Регистрация допускает крайне простые пароли (короткие, из словаря, '123456'). Нет требований к длине, сложности и проверки по базам скомпрометированных паролей, что облегчает перебор и захват аккаунтов.

3.22.2. Как использовать

Создаётся аккаунт с тривиальным паролем:

```
POST /api/Users  { "email": "x@x.op", "password": "123" }
```

Такой пароль мгновенно подбирается; в сочетании с отсутствием rate limiting это ведёт к массовому захвату учётных записей.

3.22.3. Как исправить

Рекомендуемая модель: Требования к длине/сложности + проверка по базам утечек (k-anonymity).

Допускаются тривиальные пароли. Модель: задать минимальную длину (например ≥12), запретить пароли из словарей и из баз утечек (Have I Been Pwned range API), поощрять парольные фразы и менеджеры паролей; сочетать с rate limiting и MFA.

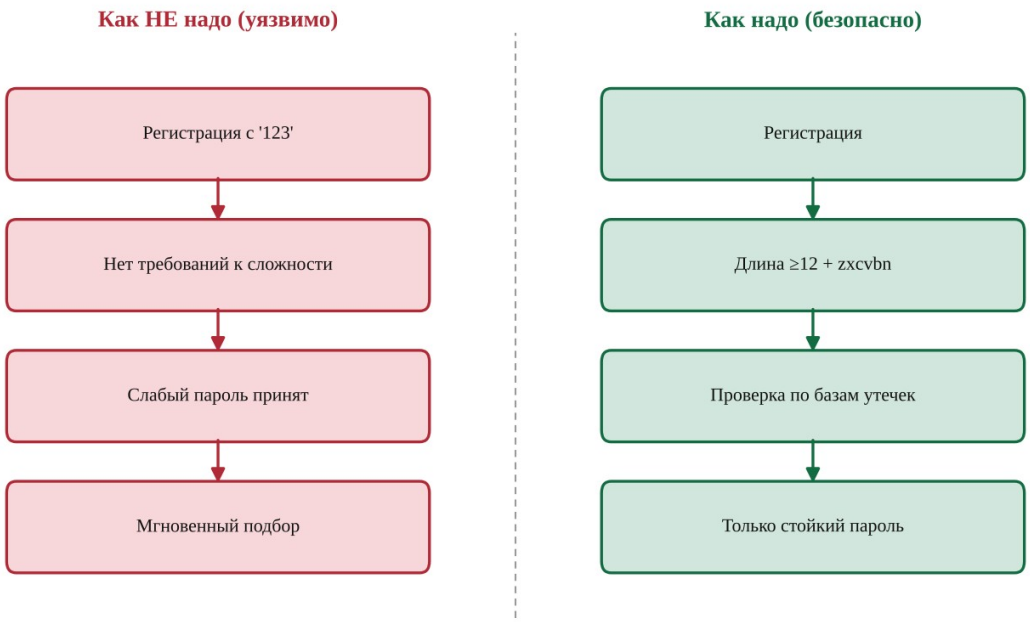


Рисунок 22 — Уязвимый и безопасный поток обработки (Слабая политика паролей)

Было (уязвимо):

```
// УЯЗВИМО: любой пароль принимается
app.post('/api/Users', (req, res) =>
  models.User.create(req.body))   // password: '123' пройдёт
```

Стало (безопасно):

```
// БЕЗОПАСНО: политика + проверка утечек
import zxcvbn from 'zxcvbn'
const pw = String(req.body.password)
if (pw.length < 12 || zxcvbn(pw).score < 3)
  return res.status(400).json({ error: 'Password too weak' })
if (await isPwned(pw))   // Have I Been Pwned (k-anonymity)
  return res.status(400).json({ error: 'Password found in breaches' })
await models.User.create({ ...req.body, password: await argon2.hash(pw) })
```

4. Заключение

Во второй части на стенде OWASP Juice Shop дополнительно обнаружено и проэксплуатировано 22 уязвимостей: межсайтовая подделка запроса и смена пароля без текущего, открытое перенаправление и небезопасный CORS, загрязнение параметров HTTP, обход клиентской и серверной защиты от XSS и

хранимый XSS через API, дефекты бизнес-логики (отрицательное количество, повторное применение купона, манипуляция балансом кошелька и бесплатное Deluxe-членство), перечисление пользователей, бессрочные JWT, предсказуемый пароль OAuth, нарушение приватности, раскрытие метрик и подробных ошибок, устаревший B2B-интерфейс, чтение произвольных файлов, отказ в обслуживании через разбор YAML и слабая парольная политика. Вместе с первой частью покрыты все десять категорий OWASP Top 10 2021.

Для каждой уязвимости показаны обнаружение, эксплуатация и устранение с конкретной моделью, кодом и схемой. Большинство проблем устраняется базовыми практиками безопасной разработки: параметризация и валидация ввода, экранирование вывода, серверный контроль доступа по ролям, безопасное хранение секретов и паролей, актуализация зависимостей и корректная конфигурация security-заголовков. Результаты подтверждают ценность OWASP Juice Shop как площадки для отработки навыков наступательной и оборонительной безопасности.

5. Список литературы

Формат: Электронный ресурс [URL] — название статьи, автор, дата обращения. Номера соответствуют надстрочным сноскам в тексте.

1. Электронный ресурс [https://owasp.org/www-project-juice-shop/] — OWASP Juice Shop Project, OWASP Foundation, дата обращения: 15.09.2026.
2. Электронный ресурс [https://pwning.owasp-juice.shop/] — Pwning OWASP Juice Shop (companion guide), Björn Kimminich, дата обращения: 15.09.2026.
3. Электронный ресурс [https://owasp.org/www-project-top-ten/] — OWASP Top 10, OWASP Foundation, дата обращения: 16.09.2026.
4. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html] — Cross-Site Request

Forgery Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 17.09.2026.

5. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html] — Unvalidated Redirects and Forwards Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 19.09.2026.
6. Электронный ресурс [https://portswigger.net/web-security/cors] — Cross-origin resource sharing (CORS), PortSwigger Web Security Academy, дата обращения: 20.09.2026.
7. Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution] — Testing for HTTP Parameter Pollution, OWASP WSTG, дата обращения: 21.09.2026.
8. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html] — Cross Site Scripting Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 23.09.2026.
9. Электронный ресурс [https://owasp.org/www-community/xss-filter-evasion-cheatsheet] — XSS Filter Evasion Cheat Sheet, OWASP Community, дата обращения: 24.09.2026.
10. Электронный ресурс [https://owasp.org/www-community/Types_of_Cross-Site_Scripting] — Types of Cross-Site Scripting, OWASP Community, дата обращения: 25.09.2026.
11. Электронный ресурс [https://owasp.org/Top10/A04_2021-Insecure_Design/] — A04:2021 – Insecure Design, OWASP Foundation, дата обращения: 27.09.2026.

12. Электронный ресурс [https://cwe.mitre.org/data/definitions/840.html] — CWE-840: Business Logic Errors, MITRE CWE, дата обращения: 28.09.2026.

13. Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/README] — Business Logic Testing, OWASP WSTG, дата обращения: 29.09.2026.

14. Электронный ресурс [https://owasp.org/Top10/A01_2021-Broken_Access_Control/] — A01:2021 – Broken Access Control, OWASP Foundation, дата обращения: 01.10.2026.

15. Электронный ресурс [https://cwe.mitre.org/data/definitions/204.html] — CWE-204: Observable Response Discrepancy, MITRE CWE, дата обращения: 02.10.2026.

16. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html] — JSON Web Token Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 03.10.2026.

17. Электронный ресурс [https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/] — A07:2021 – Identification and Authentication Failures, OWASP Foundation, дата обращения: 05.10.2026.

18. Электронный ресурс [https://cwe.mitre.org/data/definitions/359.html] — CWE-359: Exposure of Private Personal Information, MITRE CWE, дата обращения: 06.10.2026.

19. Электронный ресурс [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/README] — Information Gathering, OWASP WSTG, дата обращения: 08.10.2026.

20. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/XML_Security_Cheat_Sheet.html] — XML Security Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 09.10.2026.

21. Электронный ресурс [https://owasp.org/www-community/attacks/Path_Traversal] — Path Traversal, OWASP Community, дата обращения: 11.10.2026.

22. Электронный ресурс [https://cwe.mitre.org/data/definitions/400.html] — CWE-400: Uncontrolled Resource Consumption, MITRE CWE, дата обращения: 12.10.2026.

23. Электронный ресурс [https://cwe.mitre.org/data/definitions/620.html] — CWE-620: Unverified Password Change, MITRE CWE, дата обращения: 13.10.2026.

24. Электронный ресурс [https://cwe.mitre.org/data/definitions/209.html] — CWE-209: Generation of Error Message Containing Sensitive Information, MITRE CWE, дата обращения: 15.10.2026.

25. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html] — Authentication Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 16.10.2026.